

Maintenance Meets AI

Introduction to Neural Networks: ReneWind PGP-AIML-BA-UTA-Mar25-E

Date 07/31/2025

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- Data Overview
- EDA Results
- Data Preprocessing
- Model Evaluation Criterion
- Model Performance Summary
- Test Performance of Best Model
- Appendix

Executive Summary

- Renewind aims to develop an AI-driven predictive maintenance model to identify component failures and enable proactive maintenance. The key objectives are
 - Identify components that require maintenance to prevent failures and reduce replacement costs.
 - Minimize downtime by enabling scheduled Inspection and maintenance.
 - Optimize model performance to reduce false negatives (real failures not detected) and false positives (unnecessary inspections).
- Approach
 - False negatives result in replacement costs and downtime, while false positives incur inspection costs. Given the higher cost of replacement and downtime, the model will prioritize optimizing recall.
 - Neural network-based classification models will be trained and validated using various optimizers, dropout, and number of neurons and hidden layers.
 - Model selection will be based on recall performance on trained vs. validation datasets.
- Recommendation
 - surface “high-risk” alerts directly in your work-order system, so planners can schedule targeted inspections and maintenance.
 - build a simple dashboard showing daily risk-scores, recall/precision trends and % inspected vs. repaired.
 - Quantify savings from fewer replacements vs. the extra inspection workload
 - By operationalizing ML Model under the above recommendations, ReneWind can turn raw sensor data into a proactive maintenance engine—minimizing costs, boosting uptime and extending turbine life.

Business Problem Overview and Solution Approach

- ReneWind aims to develop a classification model to predict wind turbine generator failures using sensor data, enabling proactive maintenance and reducing costs.
- By identifying potential failures, the model will help minimize replacement costs and optimize repair schedules.
- This project involves building, tuning, and evaluating various classification models to achieve the best predictive performance.
- The goal is to reduce overall maintenance costs by accurately predicting generator failures and timely alert, help to schedule timely maintenance and Inspections routines. Machine learning model will help to optimize predictive maintenance efficiency to reduce downtime due to component failures

Considering these objectives, Identifying component need for maintenance is paramount. For classification problem Accuracy, Precision, Recall and F1 score will be evaluation criteria

- **Precision** focus to reduce False Positive or tagging component for maintenance unnecessarily and incur inspection cost
- **Recall** focus on to reduce False Negative or missing the component that need maintenance otherwise will cost replacing component and downtime
- **F1** score tries to balance both the score.

In this solution, the trade-off has been deliberately made in favor of maximizing **Recall**. The primary evaluation metric for the machine learning models is **recall**, ensuring the system identify component needed for maintenance correctly and not miss any component that need maintenance also trying to control to mis identify component need maintenance for health component to reduce inspection cost

Business Problem Overview and Solution Approach

Development of the neural network classification model will involve the following preparation steps

- Load the test and training dataset
- Perform sanity check and confirm data is loaded correctly
- Check number of records in train and test data sets
- Check sample rows from the datasets
- Perform univariate and multivariate data analysis
- Check for missing value and apply corrective steps
- Check for the duplicates and remove duplicates
- Check Outlier and evaluate features

Business Problem Overview and Solution Approach

Following steps will be performed to build and evaluate each model.

- Split data in Training, Validation and Test Dataset
- Define epoch and batch size
- Define activation function and hidden layer count
- Define on the number of neurons needed for each layer
- Define on the normalization or dropout factor to use
- Define Optimizer function to use
- Define class weights for imbalance datasets
- Check the model loss on train vs validation with number of epoch
- Build a classification report to check the model score for each class and analyze F1, Precision and Recall score
- Check Model performance On train and validation datasets

Business Problem Overview and Solution Approach

Final Model Selection:

- Compare model performance on their test and validation data sets
- Compare the model performance report for test and validation loss curve against epoch
- Compare Model performance on both classes [0, 1]

Select final model that exceeds in evaluation criteria (**Recall**) and balance performance.

EDA Results

- Training and test data is given as two separate data sets
- Training dataset got 20,000 rows and 41 columns all the input features are float data type and output or result column (target column) is integer
- Features V1 and V2 are missing 18 values each in training dataset, given their non-null counts are 19,982 out of 20,000 total rows
- V32 data has the highest spread std deviation ~ 5.5 its data range from (-19.88) to (23.63)
- V22 has the least spread with std deviation ~ 1.65 its data range from (-10.12) to (7.41)
- V21 has mean less than median and strong left skew large number of negative value pulls mean to lower
- V16 has mean less than median and left skew large number. its 75 percent values are below 0
- V38 has slight left skew. Mean is close to median, indicate tails are balanced
- Target Data is Binary classification and has only 5.5 % column have value 1 to indicate data is imbalance in both test and train data sets

[Link to Appendix slide on data background check](#)

EDA Results

- Heat map of the data gave below insight
- Most of the features are weakly correlated and range from -0.2 to + 0.2. This indicates feature have low multicollinearity and good candidate for the machine learning model building
- Some features have high correlation and may capture similar or related signal, e.g. below features have more than 0.8 correlation coefficient
 - $V7 - V15 \Rightarrow + 0.87$
 - $V8 - V16 \Rightarrow + 0.8$
 - $V11 - V29 \Rightarrow + 0.81$
 - $V16 - V21 \Rightarrow + 0.84$
 - $V24 - V32 \Rightarrow + 0.83$
- Some features have negative correlation indicates high inverse correlation and suggest some structural relations e.g.,
 - $V2 \text{ and } V14 \Rightarrow - 0.85$
 - $V3 \text{ and } V23 \Rightarrow - 0.79$
 - $V9 \text{ and } V16 \Rightarrow - 0.75$
 - $V14 \text{ and } V38 \Rightarrow - 0.76$
 - $V24 \text{ and } V30 \Rightarrow - 0.76$
 - $V28 \text{ and } V32 \Rightarrow - 0.77$

[Link to Appendix slide on data background check](#)

EDA Results

- Heat map of the data gave below insight
- Most of the features are weakly correlated and range from -0.2 to + 0.2. This indicates feature have low multicollinearity and good candidate for the machine learning model building

Some features have high correlation and may capture similar or related signal, e.g. below features have more than 0.8 correlation coefficient

- $V7 - V15 \Rightarrow + 0.87$
- $V8 - V16 \Rightarrow + 0.8$
- $V11 - V29 \Rightarrow + 0.81$
- $V16 - V21 \Rightarrow + 0.84$
- $V24 - V32 \Rightarrow + 0.83$

Some features have negative correlation indicates high inverse correlation and suggest some structural relations e.g,

- $V2 \text{ and } V14 \Rightarrow - 0.85$
- $V3 \text{ and } V23 \Rightarrow - 0.79$
- $V9 \text{ and } V16 \Rightarrow - 0.75$
- $V14 \text{ and } V38 \Rightarrow - 0.76$
- $V24 \text{ and } V30 \Rightarrow - 0.76$
- $V28 \text{ and } V32 \Rightarrow - 0.77$

[Link to Appendix slide on data background check](#)

Data Preprocessing

- No duplicate values found in test or train data
- Missing value Treatment
 - Train data v1 and v2 columns had 18 missing value each
 - Test data v1 had 5 missing value and v2 column had 6 missing values
 - To avoid data leakage, values are imputed after splitting the train data in train and validation
 - Values are imputed using simpleImputer with median strategy
- Outlier Treatment
 - All the sensor data from v1 to v40 have outlier.
 - Outlier considered as legit data as no addition information known for these variable and no changes are done to clip the data
- Neural network activation functions (e.g., sigmoid, ReLU, Tanh) produce continuous floating point outputs. Due to the nature of neural network operations and the optimization process, “Target” column binary values are converted to float to be compatible for neural networks TensorFlow/Keras lib.
- Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other.
- Interpretability of traditional ML models is compromised by multicollinearity. Neural networks can learn effective feature representations that reduce the impact of correlated variables, hence no feature engineering is done to remove some high correlation features found in the Heatmap analysis

[Link to Appendix slide on data background check](#)

Data Preprocessing

Data preparation for modeling

- The **Target** column is used as the prediction label
- **Sensor data columns V1 to V40** are used as feature inputs.
- Given training dataset is split into **training and validation sets in an 80:20 ratio**, using **stratified sampling** to maintain class distribution
- **Training data:** 16,000 rows with 40 features Class 0: 15,112 samples (94.45%) Class 1: 888 samples (5.55%)
- **Validation data:** 4,000 rows with 40 features Class 0: 3,778 samples (94.45%) Class 1: 222 samples (5.55%)
- **Test dataset:** 5,000 rows with 40 features Class 0: 4,718 samples (94.36%) Class 1: 282 samples (5.64%)
- Missing values in features **V1 and V2** were imputed using the **median strategy** across all datasets (train, validation, and test) individually to avoid data leakage
- Verified that **no missing values** remain in any of the datasets.
- Converted y_train, y_val, and y_test from **Pandas Series to NumPy arrays** for compatibility with the **TensorFlow** library.

[Link to Appendix slide on data background check](#)

Model Evaluation Criterion

Recall is the chosen evaluation criterion in this solution, based on the following considerations:

- **True Positive (TP):** Correctly predicted failure → leads to repair cost, which is acceptable and desirable.
- **False Positive (FP):** Predicted failure where there is none → results in an inspection cost, which is assumed to be minimal.
- **False Negative (FN):** Missed actual failure → leads to replacement cost, which is high and causes unplanned downtime.

Recall = $TP / (TP + FN)$ → it penalizes false negatives and rewards true positives.

Minimizing missed failures is critical, as undetected failures lead to **costly replacements**, **unplanned downtime**, and **safety risks**. Additionally, preventive maintenance improves **equipment availability**, reduces **downtime**, extends **equipment life**, saves **money and time**, and helps maintain a **safe working environment**

The model will also evaluate performance on both the **validation** and **test datasets**, in addition to the **training** set, to ensure it **generalizes well**, avoids **overfitting**, and doesn't simply capture noise.

Performance will be assessed across **both classes (0 and 1)** to confirm that the model can accurately predict the **minority class (failures)**, not just the dominant class.

Model Performance Summary

- Summary of performance metrics for training and validation data

Training set performance comparison:

	Model 0	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6
Accuracy	0.988563	0.990375	0.985062	0.988750	0.997375	0.993938	0.991062
Recall	0.908089	0.923888	0.873378	0.952706	0.977411	0.949623	0.959230
Precision	0.980633	0.982680	0.981644	0.941494	0.997503	0.992114	0.955850
F1 Score	0.941023	0.951132	0.919813	0.947020	0.987223	0.969786	0.957533

Validation set performance comparison:

	Model 0	Model 1	Model 2	Model 3	Model 4	Model 5	Model 6
Accuracy	0.986000	0.988750	0.985750	0.988000	0.990000	0.991750	0.989750
Recall	0.892953	0.911368	0.880101	0.940650	0.933229	0.934155	0.943696
Precision	0.969385	0.979010	0.981515	0.944420	0.969219	0.986035	0.957295
F1 Score	0.927347	0.942291	0.924101	0.942526	0.950411	0.958444	0.950379

[Link to Appendix slide on model assumptions](#)

Model Performance Summary

- **Model 6** (3 hidden layers, 50% dropout, class weight adjust with Adam Optimizer) is the best overall, with the **highest Recall on validation** (0.943696). Performance is consistent across training and validation, indicating **good generalization**. It has very good F1(0.950379) score compare to most of the model except Model 5 precision(0.957295) is slightly better than Model 3 but lower than rest of the models.
- **Model 5** (3 hidden layer 50% Dropout with Adam Optimizer) has the **highest F1 Score on validation** (0.958444) and best Precision (0.986035), but slightly lower Recall(0.934155) that is around ~2pp below Model 6
- **Model 4** (2 hidden layers with Adam Optimizer) shows excellent Precision(0.997503) and F1(0.987223) and Recall (0.977411) Score on training but a drop in validation Precision (0.969219), F1(0.950411), Recall (0.933229), suggesting model is **overfit** (train-recall 97.7% → val-recall 93.3%) and not generalized well. Top precision but slight recall drop vs. Model 3 and Model 6.

[Link to Appendix slide on model assumptions](#)

Model Performance Summary

- **Model 3** (3 hidden layers, 50% dropout, class weight adjust with SGD Optimizer) performs well and is stable, Recall(0.940650), Precision(0.944420) and F1(0.942526); This model performed very well on all the score. It got Big jump in recall compare to Model 0 and 1. Slightly lagged behind model 6. It has Good generalization (train $\rightarrow$ val gap $\sim$ 1pp).
- **Model 2** (2 hidden layers, 50% dropout, with SGD Optimizer) have significantly low Recall(0.880101) and F1(0.880101) Scores on both training and validation sets, indicating **underfitting** and poor suitability for this task
- **Model 1** (2 hidden layers, with SGD Optimizer) has high training Recall (0.9257) but a noticeable drop on validation (0.9180); overall performance is decent but didn't fair well with other models as recall is less than 0.93
- **Model 0** (1 hidden layers, with SGD Optimizer) has good F1(0.927347) and Precision(0.969385) score on validation but poor Recall score(0.892953), It has Solid precision but misses $\sim$ 11% of real failures. it fared well on training, but validation was not that good and seems overfitting

Model 6 got Best recall of all (captures 94% of true failures) with only a modest precision trade-off. Train $\leftrightarrow$ val gap small (1.6pp), so well-regularized. **is preferred** due to its strong balance and **best Recall**, which is the key metric for minimizing missed failures.

[Link to Appendix slide on model assumptions](#)

Model Performance Summary

Selecting Final Model

- Model 3 and Model 6 Both model generalized well as their performance are consistent on Training and Validation set
- Model 3 and Model 6 both performed very well and Model 6 slight edge as it got best recall score(0.942243)
- Model 3 got good F1(0.942526) and Precision(0.944420) score compare but slightly lower score compare to model 6
- Model 6 performed better as it got aggressive class weight strategy. And helps better failure detection
- Adam Optimizer used in Model 6 more stable and adaptive for imbalance data set compare to vanilla SGD

Considering business objective of prioritizing failure detection and overall good performance, Model 6 is overall best model that got highest recall score and very stable and comparable F1 and Precision score as well.

[Link to Appendix slide on model assumptions](#)

Best Model Test Performance

- Architecture of the best model

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)

Trainable params: 5,249 (20.50 KB)

Non-trainable params: 0 (0.00 B)

[Link to Appendix slide on model assumptions](#)

Best Model Test Performance

- Model Architecture finalized after multiple trial on training and tuning phase
- Considered various Epoch values and batch sizes. Epoch size of 150 (150 passes of entire data sets) and 500 batch size around 32 (16000/500) batches of training data set for each epoch gave good balance in training and validation loss reduction and resource consumption
- **ReLU** is preferred for hidden layers in due to its computational efficiency and ability to mitigate the vanishing gradient problem.
- **Sigmoid** Activation function is used in output layer as it is suitable for binary classification
- First **hidden Layer** Neuron size kept at **64** as general rule of thumb that hidden layer should 1–2 times of the input feature considering 40 features, 64 hidden layers worked well and gave balance performance. Added more layer with decreasing number of neurons. Increasing Neurons may help, but poses risk of overfitting, higher risk of **memorizing training data** instead of learning general patterns.
- Especially dangerous when data is **limited or noisy** (e.g. your case: 16K rows × 40 features).
- **Dropout** rate of 50% is good starting point and worked well for regularization
- **SGD** and **Adam** Optimizer tried, but Adam Optimizer is selected based on the results and also Adam optimizer performed well due to its adaptive learning rate adjustment, leading to faster convergence and better performance

[Link to Appendix slide on model assumptions](#)

Best Model Test Performance

Network Architecture

- **Input Layer:** Accepts 40 features (sensor data V1 to V40)
- **Hidden Layer 1:** Dense layer with **64 neurons**, activation = **ReLU**
- **Dropout Layer:** **50% dropout rate** applied after the first hidden layer for regularization
- **Hidden Layer 2:** Dense layer with **32 neurons**, activation = **ReLU**
- **Hidden Layer 3:** Dense layer with **16 neurons**, activation = **ReLU**
- **Output Layer:** Dense layer with **1 neuron**, activation = **sigmoid** (for binary classification)

Training Configuration

- **Epochs:** 150 and **Batch Size:** 500
- **Optimizer:** **Adam**, chosen for its adaptive learning rate and strong performance on noisy datasets
- **Loss Function:** **Binary Cross Entropy**, suitable for binary classification tasks
- **Class Weights:** {0: 1, 1: 20} — used to **address class imbalance**, ensuring failures (class 1) are given more importance

[Link to Appendix slide on model assumptions](#)

Best Model Test Performance

- **Recall** was the primary evaluation criterion for model selection.
- **Model 6** achieved the **recall score** of (0.936961) on the test dataset.
- It also performed well on **Accuracy(0.988)**, **F1** (0.942867), and precision (0.948958) slightly lower than **Validation data set but**
- This results indicate model is tuned properly and generalized well, no overfitting

While the model performs exceptionally well on **class 0**, the **recall for class 1 is 0.88**, indicating a few failures are still missed (false negatives). However, this is still a **strong performance** on an imbalanced dataset and aligns well with the business goal of prioritizing failure detection.

Best Model Performnace on Test data set

	Accuracy	Recall	Precision	F1 Score
0	0.988	0.936961	0.948958	0.942867

Best Model Performnace on Test data set

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	4718
1.0	0.91	0.88	0.89	282
accuracy			0.99	5000
macro avg	0.95	0.94	0.94	5000
weighted avg	0.99	0.99	0.99	5000

[Link to Appendix slide on model assumptions](#)

APPENDIX

Data background check

- Sample Rows from training data set

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33	V34	V35	V36	V37	V38	V39	V40	Target
0	-4.464606	-4.679129	3.101546	0.506130	-0.221083	-2.032511	-2.910870	0.050714	-1.522351	3.761892	...	3.059700	-1.690440	2.846296	2.235198	6.667486	0.443809	-2.369169	2.950578	-3.480324	0
1	3.365912	3.653381	0.909671	-1.367528	0.332016	2.358938	0.732600	-4.332135	0.565695	-0.101080	...	-1.795474	3.032780	-2.467514	1.894599	-2.297780	-1.731048	5.908837	-0.386345	0.616242	0
2	-3.831843	-5.824444	0.634031	-2.418815	-1.773827	1.016824	-2.098941	-3.173204	-2.081860	5.392621	...	-0.257101	0.803550	4.086219	2.292138	5.360850	0.351993	2.940021	3.839160	-4.309402	0
3	1.618098	1.888342	7.046143	-1.147285	0.083080	-1.529780	0.207309	-2.493629	0.344926	2.118578	...	-3.584425	-2.577474	1.363769	0.622714	5.550100	-1.526796	0.138853	3.101430	-1.277378	0
4	-0.111440	3.872488	-3.758361	-2.982897	3.792714	0.544960	0.205433	4.848994	-1.854920	-6.220023	...	8.265896	6.629213	-10.068689	1.222987	-3.229763	1.686909	-2.163896	-3.644622	6.510338	0

5 rows x 41 columns

- Sample Rows from test data set

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33	V34	V35	V36	V37	V38	V39	V40	Target
0	-0.613489	-3.819640	2.202302	1.300420	-1.184929	-4.495964	-1.835817	4.722989	1.206140	-0.341909	...	2.291204	-5.411388	0.870073	0.574479	4.157191	1.428093	-10.511342	0.454664	-1.448363	0
1	0.389608	-0.512341	0.527053	-2.576776	-1.016766	2.235112	-0.441301	-4.405744	-0.332869	1.966794	...	-2.474936	2.493582	0.315165	2.059288	0.683859	-0.485452	5.128350	1.720744	-1.488235	0
2	-0.874861	-0.640632	4.084202	-1.590454	0.525855	-1.957592	-0.695367	1.347309	-1.732348	0.466500	...	-1.318888	-2.997464	0.459664	0.619774	5.631504	1.323512	-1.752154	1.808302	1.675748	0
3	0.238384	1.458607	4.014528	2.534478	1.196987	-3.117330	-0.924035	0.269493	1.322436	0.702345	...	3.517918	-3.074085	-0.284220	0.954576	3.029331	-1.367198	-3.412140	0.906000	-2.450889	0
4	5.828225	2.768260	-1.234530	2.809264	-1.641648	-1.406698	0.568643	0.965043	1.918379	-2.774855	...	1.773841	-1.501573	-2.226702	4.776830	-6.559698	-0.805551	-0.276007	-3.858207	-0.537694	0

Data background check

- Train and test Data got 42 columns Target is prediction columns and is integer columns other 40 columns are sensor data and float data type.

Train = 20000 records

#	Column	Non-Null Count	Dtype
0	V1	19982 non-null	float64
1	V2	19982 non-null	float64
2	V3	20000 non-null	float64
3	V4	20000 non-null	float64
4	V5	20000 non-null	float64
5	V6	20000 non-null	float64
6	V7	20000 non-null	float64
7	V8	20000 non-null	float64
8	V9	20000 non-null	float64
9	V10	20000 non-null	float64
10	V11	20000 non-null	float64
11	V12	20000 non-null	float64
12	V13	20000 non-null	float64
13	V14	20000 non-null	float64
14	V15	20000 non-null	float64
15	V16	20000 non-null	float64
16	V17	20000 non-null	float64
17	V18	20000 non-null	float64
18	V19	20000 non-null	float64
19	V20	20000 non-null	float64
20	V21	20000 non-null	float64
21	V22	20000 non-null	float64
22	V23	20000 non-null	float64
23	V24	20000 non-null	float64
24	V25	20000 non-null	float64
25	V26	20000 non-null	float64
26	V27	20000 non-null	float64
27	V28	20000 non-null	float64
28	V29	20000 non-null	float64
29	V30	20000 non-null	float64
30	V31	20000 non-null	float64
31	V32	20000 non-null	float64
32	V33	20000 non-null	float64
33	V34	20000 non-null	float64
34	V35	20000 non-null	float64
35	V36	20000 non-null	float64
36	V37	20000 non-null	float64
37	V38	20000 non-null	float64
38	V39	20000 non-null	float64
39	V40	20000 non-null	float64
40	Target	20000 non-null	int64

dtypes: float64(40), int64(1)

Test = 5000 records

#	Column	Non-Null Count	Dtype
0	V1	4995 non-null	float64
1	V2	4994 non-null	float64
2	V3	5000 non-null	float64
3	V4	5000 non-null	float64
4	V5	5000 non-null	float64
5	V6	5000 non-null	float64
6	V7	5000 non-null	float64
7	V8	5000 non-null	float64
8	V9	5000 non-null	float64
9	V10	5000 non-null	float64
10	V11	5000 non-null	float64
11	V12	5000 non-null	float64
12	V13	5000 non-null	float64
13	V14	5000 non-null	float64
14	V15	5000 non-null	float64
15	V16	5000 non-null	float64
16	V17	5000 non-null	float64
17	V18	5000 non-null	float64
18	V19	5000 non-null	float64
19	V20	5000 non-null	float64
20	V21	5000 non-null	float64
21	V22	5000 non-null	float64
22	V23	5000 non-null	float64
23	V24	5000 non-null	float64
24	V25	5000 non-null	float64
25	V26	5000 non-null	float64
26	V27	5000 non-null	float64
27	V28	5000 non-null	float64
28	V29	5000 non-null	float64
29	V30	5000 non-null	float64
30	V31	5000 non-null	float64
31	V32	5000 non-null	float64
32	V33	5000 non-null	float64
33	V34	5000 non-null	float64
34	V35	5000 non-null	float64
35	V36	5000 non-null	float64
36	V37	5000 non-null	float64
37	V38	5000 non-null	float64
38	V39	5000 non-null	float64
39	V40	5000 non-null	float64
40	Target	5000 non-null	int64

Data background check

- Statistical summary of the data v1 to V20

	count	mean	std	min	25%	50%	75%	max
V1	19982.0	-0.271996	3.441625	-11.876451	-2.737146	-0.747917	1.840112	15.493002
V2	19982.0	0.440430	3.150784	-12.319951	-1.640674	0.471536	2.543967	13.089269
V3	20000.0	2.484699	3.388963	-10.708139	0.206860	2.255786	4.566165	17.090919
V4	20000.0	-0.083152	3.431595	-15.082052	-2.347660	-0.135241	2.130615	13.236381
V5	20000.0	-0.053752	2.104801	-8.603361	-1.535607	-0.101952	1.340480	8.133797
V6	20000.0	-0.995443	2.040970	-10.227147	-2.347238	-1.000515	0.380330	6.975847
V7	20000.0	-0.879325	1.761626	-7.949681	-2.030926	-0.917179	0.223695	8.006091
V8	20000.0	-0.548195	3.295756	-15.657561	-2.642665	-0.389085	1.722965	11.679495
V9	20000.0	-0.016808	2.160568	-8.596313	-1.494973	-0.067597	1.409203	8.137580
V10	20000.0	-0.012998	2.193201	-9.853957	-1.411212	0.100973	1.477045	8.108472

	count	mean	std	min	25%	50%	75%	max
V11	20000.0	-1.895393	3.124322	-14.832058	-3.922404	-1.921237	0.118906	11.826433
V12	20000.0	1.604825	2.930454	-12.948007	-0.396514	1.507841	3.571454	15.080698
V13	20000.0	1.580486	2.874658	-13.228247	-0.223545	1.637185	3.459886	15.419616
V14	20000.0	-0.950632	1.789651	-7.738593	-2.170741	-0.957163	0.270677	5.670664
V15	20000.0	-2.414993	3.354974	-16.416606	-4.415322	-2.382617	-0.359052	12.246455
V16	20000.0	-2.925225	4.221717	-20.374158	-5.634240	-2.682705	-0.095046	13.583212
V17	20000.0	-0.134261	3.345462	-14.091184	-2.215611	-0.014580	2.068751	16.756432
V18	20000.0	1.189347	2.592276	-11.643994	-0.403917	0.883398	2.571770	13.179863
V19	20000.0	1.181808	3.396925	-13.491784	-1.050168	1.279061	3.493299	13.237742
V20	20000.0	0.023608	3.669477	-13.922659	-2.432953	0.033415	2.512372	16.052339

Data background check

- Statistical summary of the data v21 to V40 and target

	count	mean	std	min	25%	50%	75%	max
V21	20000.0	-3.611252	3.567690	-17.956231	-5.930360	-3.532888	-1.265884	13.840473
V22	20000.0	0.951835	1.651547	-10.122095	-0.118127	0.974687	2.025594	7.409856
V23	20000.0	-0.366116	4.031860	-14.866128	-3.098756	-0.262093	2.451750	14.458734
V24	20000.0	1.134389	3.912069	-16.387147	-1.468062	0.969048	3.545975	17.163291
V25	20000.0	-0.002186	2.016740	-8.228266	-1.365178	0.025050	1.397112	8.223389
V26	20000.0	1.873785	3.435137	-11.834271	-0.337863	1.950531	4.130037	16.836410
V27	20000.0	-0.612413	4.368847	-14.904939	-3.652323	-0.884894	2.189177	17.560404
V28	20000.0	-0.883218	1.917713	-9.269489	-2.171218	-0.891073	0.375884	6.527643
V29	20000.0	-0.985625	2.684365	-12.579469	-2.787443	-1.176181	0.629773	10.722055
V30	20000.0	-0.015534	3.005258	-14.796047	-1.867114	0.184346	2.036229	12.505812

	count	mean	std	min	25%	50%	75%	max
V31	20000.0	0.486842	3.461384	-13.722760	-1.817772	0.490304	2.730688	17.255090
V32	20000.0	0.303799	5.500400	-19.876502	-3.420469	0.052073	3.761722	23.633187
V33	20000.0	0.049825	3.575285	-16.898353	-2.242857	-0.066249	2.255134	16.692486
V34	20000.0	-0.462702	3.183841	-17.985094	-2.136984	-0.255008	1.436935	14.358213
V35	20000.0	2.229620	2.937102	-15.349803	0.336191	2.098633	4.064358	15.291065
V36	20000.0	1.514809	3.800860	-14.833178	-0.943809	1.566526	3.983939	19.329576
V37	20000.0	0.011316	1.788165	-5.478350	-1.255819	-0.128435	1.175533	7.467006
V38	20000.0	-0.344025	3.948147	-17.375002	-2.987638	-0.316849	2.279399	15.289923
V39	20000.0	0.890653	1.753054	-6.438880	-0.272250	0.919261	2.057540	7.759877
V40	20000.0	-0.875630	3.012155	-11.023935	-2.940193	-0.920806	1.119897	10.654265
Target	20000.0	0.055500	0.228959	0.000000	0.000000	0.000000	0.000000	1.000000

Data background check

- Missing values in train data are treated with median strategy in train and val splits

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33	V34	V35	V36	V37	V38	V39	V40	Target
89	NaN	-3.961403	2.787804	-4.712526	-3.007329	-1.541245	-0.881148	1.476656	0.574700	-1.100884	...	-8.326069	-5.140552	1.121314	-0.305907	5.315007	3.750044	-5.631174	2.372485	2.195956	0.0
613	-2.048681	NaN	1.623885	-3.324224	0.152256	0.600157	-1.812802	0.852194	-1.522600	0.211071	...	3.264218	2.379064	-1.257084	1.719365	2.537010	1.701780	-1.434535	0.597365	0.739238	0.0
2236	-3.760658	NaN	0.194954	-1.637958	1.261479	-1.573947	-3.685700	1.575651	-0.309823	-0.137656	...	7.620821	1.695061	-3.956354	2.707644	4.657387	1.619307	-5.537285	1.246650	-1.162793	0.0
2508	-1.430888	NaN	0.659576	-2.876402	1.150137	-0.785760	-1.560174	2.898635	-2.346989	-0.217607	...	6.279266	3.323914	-4.047760	3.119220	3.336260	0.603524	-3.781725	-0.157478	1.503298	0.0
4653	5.465769	NaN	4.540947	-2.916550	0.399752	2.798925	0.029477	-7.334071	1.122674	1.695269	...	-1.535753	4.596212	-4.103525	4.295524	0.152672	-3.726700	0.656292	0.704652	-0.461696	0.0
5941	NaN	1.008391	1.122702	5.397082	0.064230	-2.706919	-2.028368	0.534046	3.006797	-2.362238	...	1.869502	-3.115298	-0.550197	1.713781	-2.256960	0.410992	-3.434400	-1.299388	-1.768734	0.0
6317	NaN	-5.205346	1.997652	-3.707913	-1.042200	-1.593126	-2.653309	0.852280	-1.310489	2.406924	...	3.074149	-0.067469	-0.077521	3.196840	7.016205	1.302334	-4.580096	2.956254	-2.363150	0.0
6464	NaN	2.146202	5.004415	4.192063	1.427887	-6.436263	-0.931339	3.794120	-0.683032	-0.738941	...	5.231472	-5.113312	1.745687	2.587189	3.990777	0.610716	-4.273457	1.864566	-3.599079	0.0
6810	-2.631454	NaN	2.330188	1.090080	0.603973	-1.139383	-0.690121	-1.358935	0.355568	-1.189176	...	-0.950215	0.209717	0.448728	1.046063	0.536937	0.763187	1.728621	1.885821	-1.701774	0.0
7073	NaN	2.534010	2.762821	-1.673718	-1.942214	-0.029961	0.911323	-3.199743	2.948610	-0.413229	...	-4.887077	-2.611526	-1.500807	2.036186	-0.828979	-1.369591	0.572366	-0.132183	-0.322007	0.0
7788	-4.203459	NaN	2.953868	0.584466	4.103940	-0.639211	-2.810799	-0.112492	-1.362768	-0.800101	...	12.522374	9.502488	-7.152953	5.668769	1.249833	-2.158520	-0.954461	-0.002385	-1.546808	0.0
8431	NaN	-1.398710	-2.008106	-1.750341	0.932279	-1.290327	-0.270476	4.458834	-2.776270	-1.211766	...	4.560244	-0.420834	-2.037313	1.109793	1.520594	2.113872	-2.252571	-0.939249	2.542411	0.0
8439	NaN	-3.840585	0.197220	4.147789	1.151400	-0.993298	-4.732363	0.558966	-0.926683	0.457914	...	6.818725	3.451213	0.241818	3.215765	1.203210	1.274857	-1.921229	0.578890	-2.837521	0.0
8483	-4.484232	NaN	1.200644	-2.042068	2.779443	-0.801748	-6.403546	-1.225314	-1.485831	-0.974362	...	9.467401	4.281421	-7.588117	3.266825	5.232311	1.278590	-5.377013	1.984130	-1.643391	0.0
8894	3.263555	NaN	8.448574	-3.253218	-3.417978	-2.995938	-0.668271	-0.161283	-0.666870	3.133527	...	-4.242730	-3.122680	2.522415	5.283805	7.291310	-0.867555	-4.315230	3.124488	-2.393239	0.0
8947	-3.793170	NaN	0.719610	2.306296	0.934728	-0.984321	0.504807	-0.441008	-2.787177	1.734671	...	1.527720	-0.498910	3.789730	1.130689	0.618278	-0.111146	5.708912	1.542366	-2.481019	0.0
9362	2.662045	NaN	2.980068	4.430762	-0.237769	0.671919	0.380068	-7.646684	4.434754	-0.746393	...	5.493590	-1.104656	1.224987	0.975596	-4.794411	-2.269039	7.670648	0.824983	-3.929104	0.0
9425	-2.354134	NaN	2.053893	0.811660	2.540366	-0.924875	-0.208380	-0.558284	-0.140210	-2.146916	...	-0.621103	-0.896509	-1.181480	-1.236617	1.237120	1.238277	2.073727	1.472175	0.0	
9848	-1.763501	NaN	2.845012	-2.753083	-0.811848	-0.101166	-1.382141	-1.105042	-0.054339	0.159742	...	-2.158880	1.859682	-0.337278	1.509300	3.408411	0.922594	-1.502959	2.514666	-0.793574	0.0
11156	NaN	-0.666978	3.715829	4.934000	1.667596	-4.356097	-2.823137	0.373175	-0.709951	2.177428	...	6.663446	-2.897697	3.068461	2.486862	4.808548	0.069305	-1.215784	3.013674	-5.972586	0.0

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33	V34	V35	V36	V37	V38	V39	V40	Target
9362	2.662045	NaN	2.980068	4.430762	-0.237769	0.671919	0.380068	-7.646684	4.434754	-0.746393	...	-5.493590	-1.104656	1.224987	0.975596	-4.794411	-2.269039	7.670648	0.824983	-3.929104	0.0
9425	-2.354134	NaN	2.053893	0.811660	2.540366	-0.924875	-0.208380	-0.562864	-0.140210	-2.146916	...	-0.621103	-0.896509	-1.181480	-1.236617	1.237120	1.228277	2.073727	1.223874	1.472175	0.0
9848	-1.763501	NaN	2.845012	-2.753083	-0.811848	-0.101166	-1.382141	-1.105042	-0.054339	0.159742	...	-2.158880	1.859682	-0.337278	1.509300	3.408411	0.922594	-1.502959	2.514666	-0.793574	0.0
11156	NaN	-0.666978	3.715829	4.934000	1.667596	-4.356097	-2.823137	0.373175	-0.709951	2.177428	...	6.663446	-2.897697	3.068461	2.486862	4.808548	0.069305	-1.215784	3.013674	-5.972586	0.0
11287	NaN	-2.561519	-0.180836	-7.194814	-1.043832	1.384845	1.306093	1.559192	-2.929173	1.274543	...	-2.531655	0.560392	-1.153884	-0.019205	4.065248	0.978880	-0.571288	0.630374	3.191467	0.0
11456	NaN	1.299595	4.382858	1.583219	-0.076564	0.658770	-1.638530	-4.814763	-0.914819	2.811808	...	1.772287	5.755242	1.203739	5.663939	0.413630	-2.643934	5.529745	2.104536	-4.945350	0.0
11637	-2.270541	NaN	1.710061	1.157522	-0.355177	-5.449480	-0.786321	3.936176	-1.576138	0.800881	...	2.651480	-8.429332	3.511387	1.500102	5.552380	2.588580	-3.453418	3.234339	-2.760081	0.0
12221	NaN	-2.326319	-0.051978	0.616063	-0.895755	-2.437003	0.349826	2.092611	-2.933523	2.291272	...	0.134995	-5.183424	5.251667	0.716371	3.210930	1.641985	1.543559	1.805163	-2.039510	0.0
12339	-1.663687	NaN	-0.712286	-4.346935	1.391670	-0.093951	-2.163175	-0.805730	0.031191	-0.658845	...	0.306588	-2.690990	-1.311879	-1.596402	5.821108	3.462033	-1.736752	2.291092	2.204769	0.0
12447	NaN	0.752613	-0.271099	1.301204	2.038697	-1.485203	-0.411939	0.980629	-0.110336	-0.065120	...	4.410397	-2.208567	-1.358706	-1.725697	1.679060	-0.208564	-2.335547	0.112248	-0.542931	0.0
13086	NaN	2.056243	3.330642	2.741497	2.783166	-0.444191	-2.015376	-0.887154	-1.110920	0.025289	...	5.112126	4.675408	-1.709632	4.29762	0.996644	-1.190509	2.707054	0.511023	-0.884200	0.0
13411	NaN	2.704511	4.587169	1.867930	2.050133	-0.925076	-1.669496	-1.653803	-0.243383	-0.317316	...	2.527207	3.625279	-1.200200	2.328028	1.666937	-0.943228	0.946846	1.655145	-1.665439	0.0
14202	NaN	0.738653	2.144536	-3.201788	4.112972	3.375972	-1.337179	-4.546371	1.941427	-5.466593	...	0.157778	9.768106	-10.258190	0.513864	-1.974958	-0.029436	3.127488	0.009482	4.538125	0.0
15520	NaN	1.382556	3.236896	-3.818363	-1.917264	0.437686	1.347540	-0.236067	1.155712	0.306502	...	-5.414599	-0.896510	-1.057864	1.417365	1.161990	-1.147123	-0.048258	0.604532	0.814557	0.0
15913	0.768122	NaN	5.296110	0.043018	-1.173729	-2.248575	0.956395	-0.089941	-0.241678	-1.061413	...	-7.720265	-4.518617	3.182253	0.453452	2.175494	1.261707	0.892630	0.026732	0.632903	0.0
16576	NaN	3.933815	-0.761930	2.655189	1.753614	-0.554092	1.829107	-0.105409	-3.737081	1.036776	...	3.486408	1.028094	2.845747	1.744060	-1.999615	-0.783041	8.698449	0.352489	-2.005397	0.0
18104	NaN	1.492173	2.659206	-0.222784	-0.303648	-1.347322	0.443039	-0.159095	-1.108116	-0.572670	...	-1.007343	-2.229579	-0.870845	1.299595	0.667952	-0.503349	-1.485419	-0.157322	0.156501	0.0
18342	-0.928572	NaN	2.375506	-1.236914	3.228744	-2.100088	-2.189908	0.588644	1.95973	0.500849	...	1.613181	-1.820569	-6.664808	-0.455080	3.054891	2.935276	-3.791135	0.860311	3.335753	0.0
18343	-2.377369	NaN	-0.009173	-1.471979	1.295482	0.724894	-1.122797	-3.190475	3.250575	-4.861648	...	-5.093149	0.439355	-3.167241	-2.713266	-0.592845	3.229219	1.315635	2.282838	1.151589	0.0
18907	-0.119181	NaN	3.657612	-1.231802	1.946873	-0.119089	0.652414	-1.490208	-0.033631	-2.556604	...	-4.670353	-0.593916	-1.650592	-1.405071	1.531267	1.079147	2.832949	1.450781	3.323659	0.0

Data background check

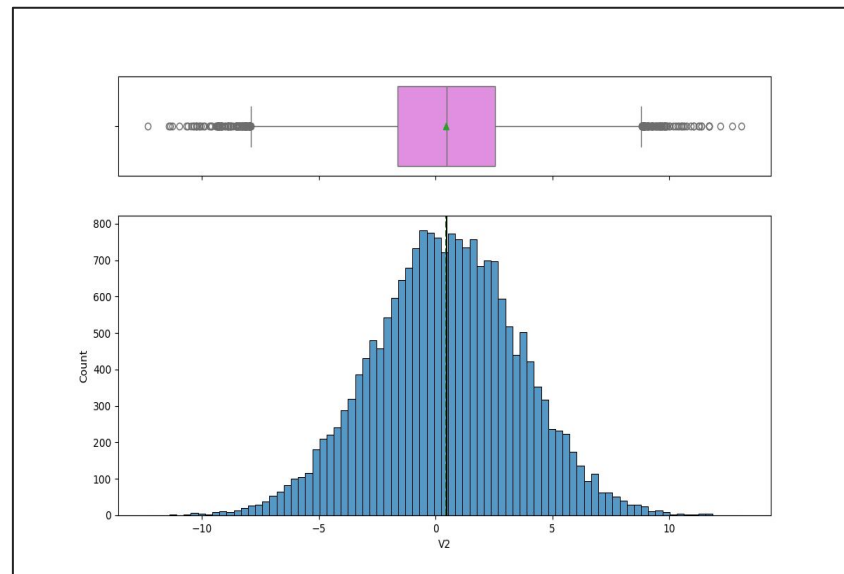
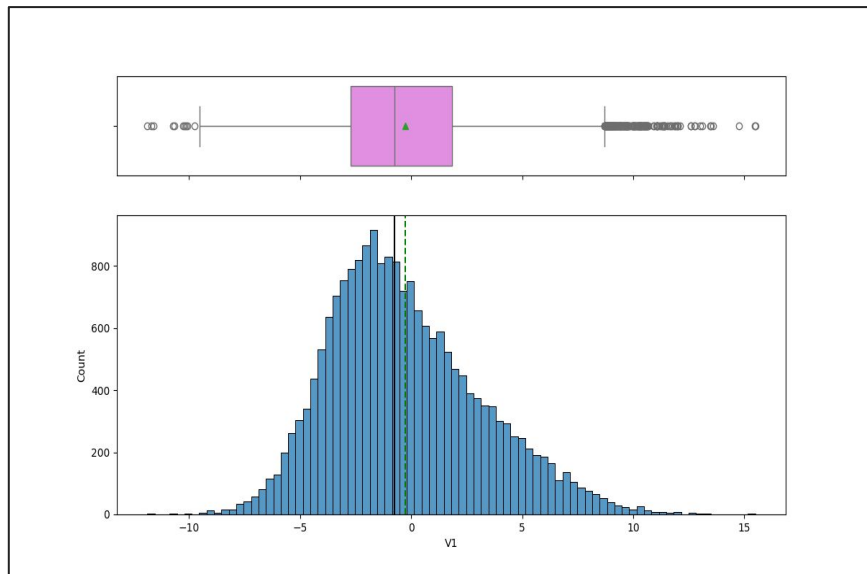
- Missing values in Test data data are treated with median strategy in train and val splits

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V32	V33	V34	V35	V36	V37	V38	V39	V40	Target
709	3.171300	NaN	-0.899604	-7.687193	-1.844379	2.229502	0.649609	0.680742	-0.079613	-3.926869	...	-8.569958	1.198974	-3.747194	-0.834087	0.364598	3.687177	-1.450631	-0.012682	6.569833	0.0
859	NaN	1.481190	2.208128	-2.550029	1.526045	-0.964918	0.559579	3.004337	-3.937734	0.105693	...	5.093845	2.920339	-3.080601	3.750633	2.422388	-0.692277	-0.182557	-0.709241	2.498946	0.0
1070	NaN	1.222743	7.023517	-1.227970	-3.385548	-1.500321	-0.375947	-2.898488	3.016750	0.097183	...	-6.401373	-2.539566	-0.427894	4.971210	1.229448	-1.620576	-2.472413	0.692331	-1.427785	0.0
1639	NaN	-5.280584	1.695313	-0.787160	-1.872912	-0.469312	-1.970327	-2.099606	-1.573940	4.521625	...	-1.939828	-2.575771	5.279322	1.557176	5.542348	1.058056	1.696663	3.691808	-3.802066	0.0
1777	1.255877	NaN	1.123121	0.347719	-0.199314	0.542522	-0.904536	-2.398356	0.228689	0.245489	...	0.851358	1.657839	-1.410919	3.587088	-1.116910	-0.865736	2.766820	-0.368560	-0.864084	0.0
1832	NaN	-0.558554	5.315575	1.517019	-2.304035	-1.410233	-1.974341	-3.081827	1.762233	1.134216	...	-3.017435	-0.475546	1.987185	4.541473	1.335494	-0.812582	-0.545365	1.922588	-4.117640	0.0
1869	-1.272832	NaN	4.426359	-3.013970	-1.294693	-0.883173	-1.731633	0.098774	-0.991360	2.495756	...	-0.396983	1.190134	0.629071	2.411258	6.166668	-0.140616	-4.208798	2.623088	-1.368893	0.0
2741	-2.938927	NaN	2.913242	1.431121	4.003345	-4.743048	-2.450111	3.795883	-0.339877	-2.897700	...	12.077656	0.671770	-6.354040	3.887011	3.420416	0.506994	-5.913055	0.214129	-0.931294	0.0
3266	5.896134	NaN	7.342806	-1.052112	-1.393952	-0.410402	0.392391	-6.141263	2.100145	1.897655	...	-5.652135	-1.973205	0.275744	3.894656	2.108591	-2.803778	3.971349	2.233942	-2.542753	0.0
4051	NaN	3.983783	0.524783	-4.776552	2.590121	1.040410	3.097642	-1.744755	-0.269377	0.410768	...	-4.134022	-5.444258	-1.925177	-5.736453	4.155637	0.047600	3.864513	1.224684	4.916014	0.0
4186	5.034513	NaN	4.450708	-6.077425	0.445417	2.491588	1.958447	-5.311945	-1.397204	2.755499	...	-5.133693	1.139969	-1.421471	0.822456	4.099736	-2.152178	7.063377	2.377923	1.906096	0.0

11 rows x 41 columns

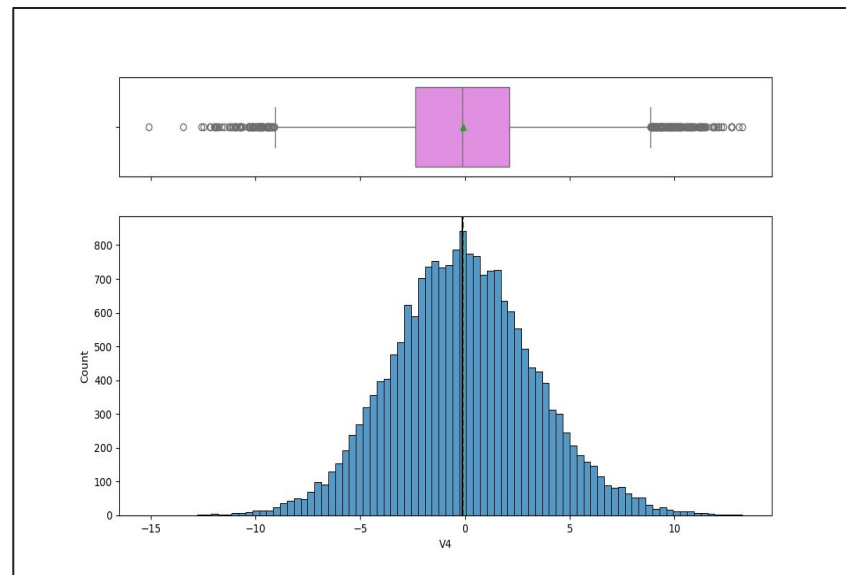
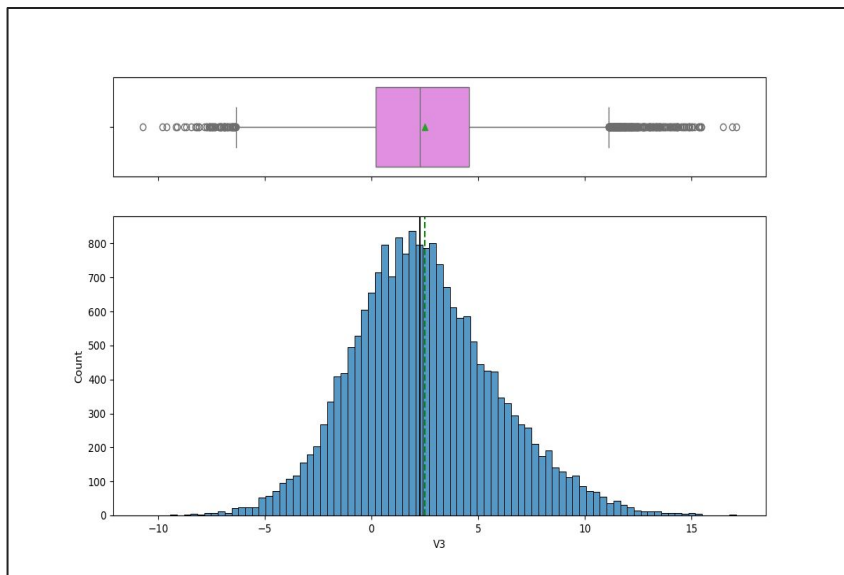
Data background check

- V1 : slightly right skewed and Data range between -12, 15.49 and mean ~ -0.27 data concentrated between -2.74, 1.84
- V2 : balanced data set is zero skewed. Data range between -12, 13.09 and mean ~ 0.44 data concentrated between -1.64, 2.54



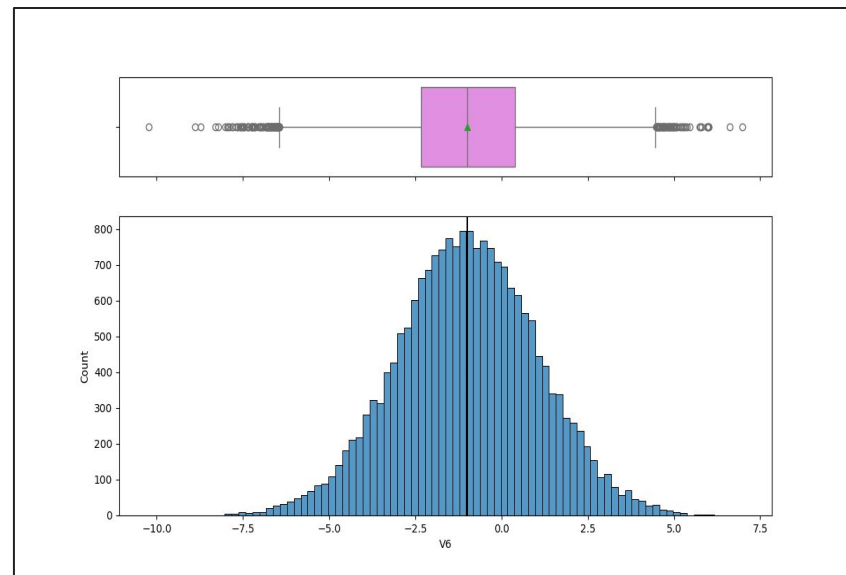
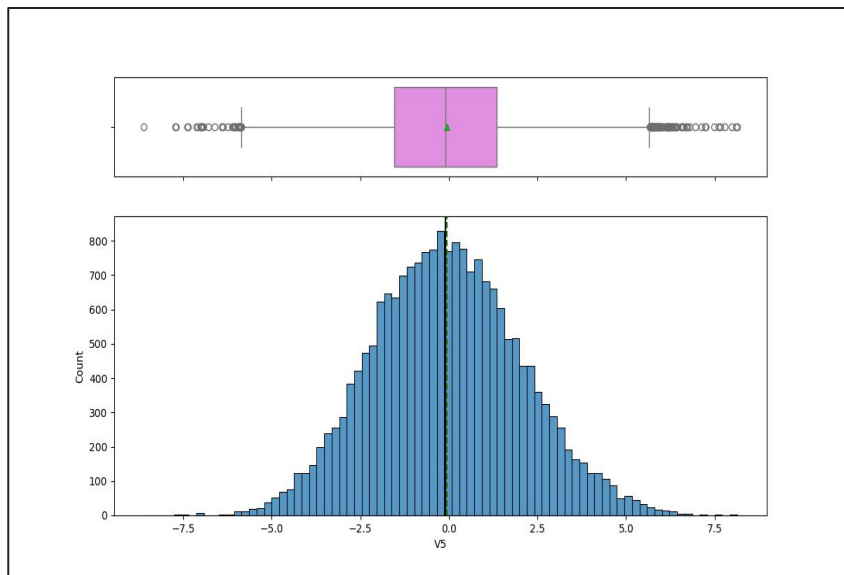
Data background check

- V3 : slightly right skewed and Data range between -11, 17.09 and mean ~ 2.48 data concentrated between 0.21, 4.57
- V4 : balanced data set is zero skewed and Data range between -15, 13.24 and mean ~ -0.08 data concentrated between -2.35, 2.13



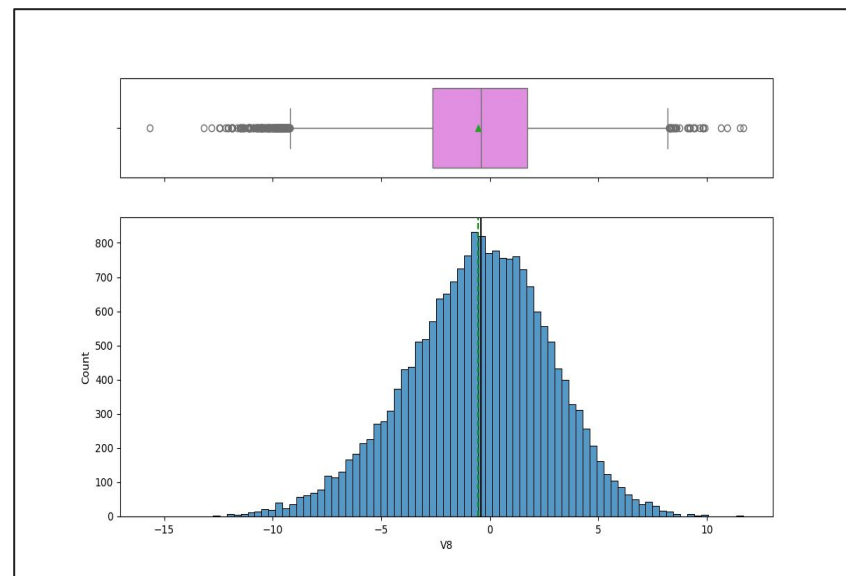
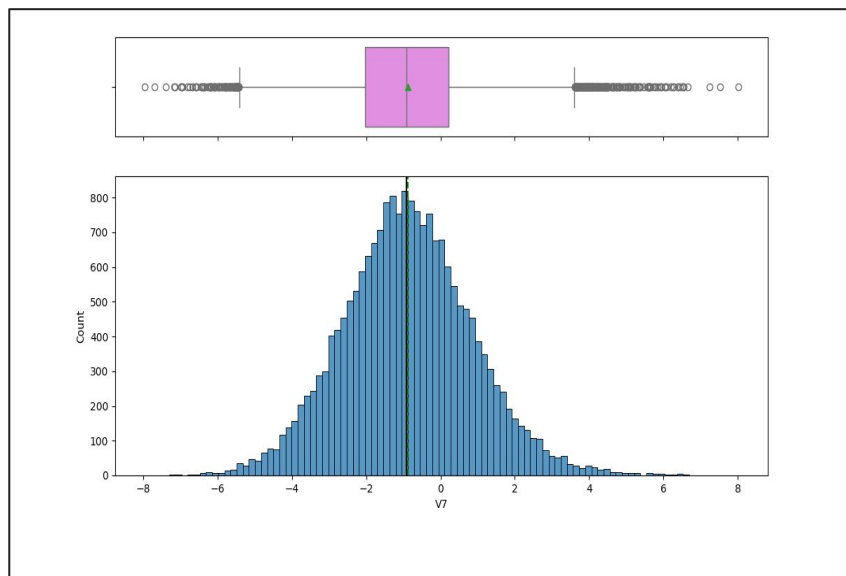
Data background check

- V5 : balanced data set has zero skew. Data range between -9, 8.13 and mean ~ -0.05 data concentrated between -1.54, 1.34
- V6 : balanced data set has zero skew. Data range between -10, 6.98 and mean ~ -1 data concentrated between -2.35, 0.38



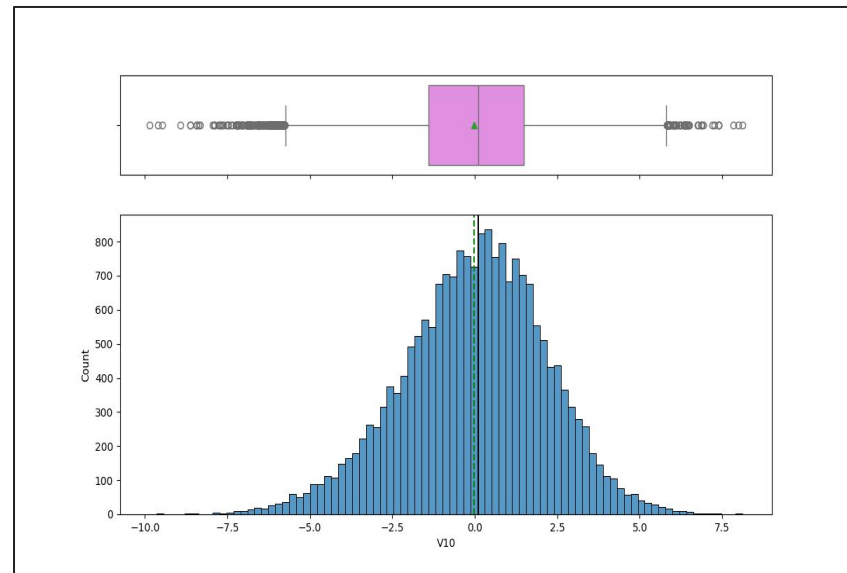
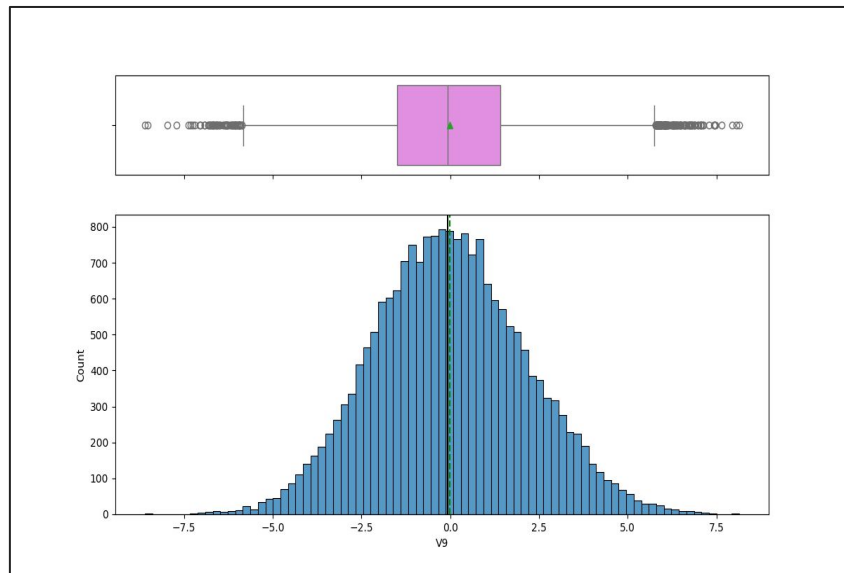
Data background check

- V7 : balanced data set has zero skew. Data range between -8, 8.01 and mean ~ -0.88 data concentrated between -2.03, 0.22
- V8 : slight left skewed. Data range between -16, 11.68 and mean ~ -0.55 data concentrated between -2.64, 1.72



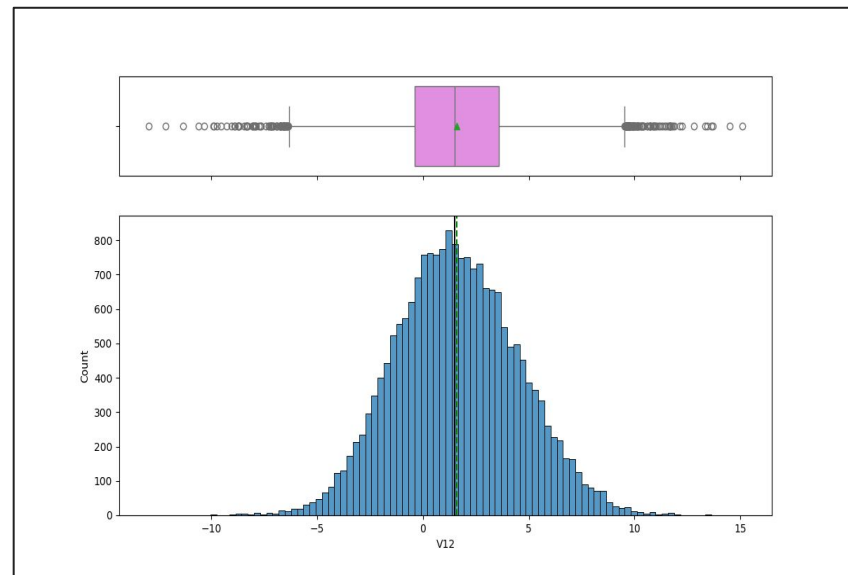
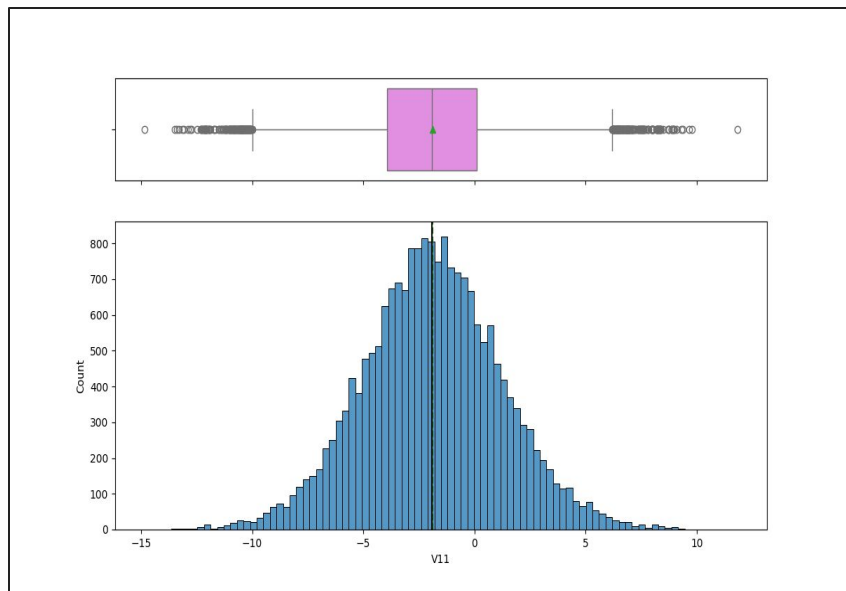
Data background check

- V9 : balanced data set has zero skew. Data range between -9, 8.14 and mean ~ -0.02 data concentrated between -1.49, 1.41
- V10 : slight left skewed. Data range between -10, 8.11 and mean ~ -0.01 data concentrated between -1.41, 1.48



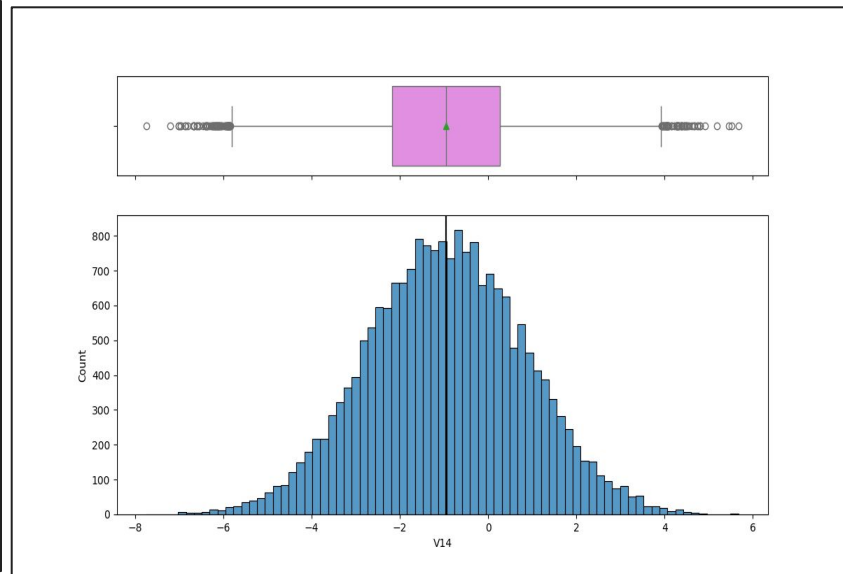
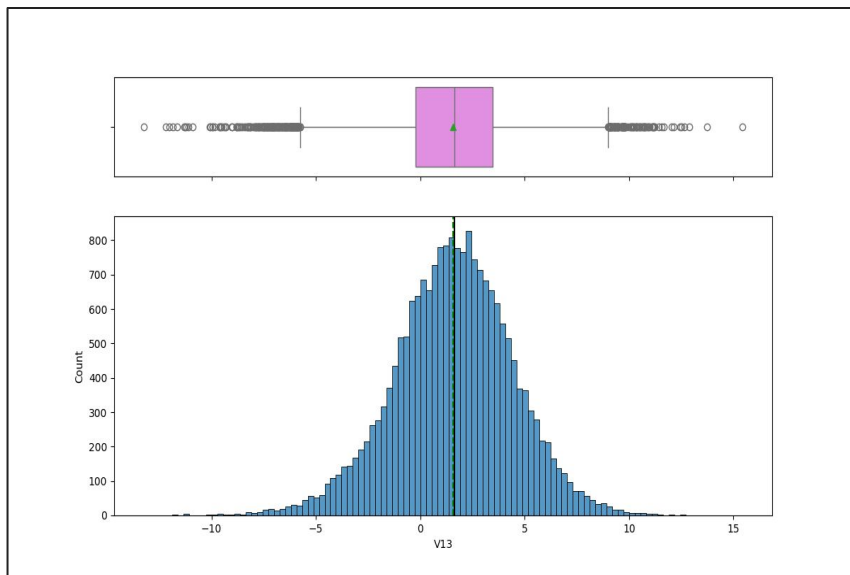
Data background check

- V11 : balance data with 0 skew and Data range between -15, 11.83 and mean ~ -1.9 data concentrated between -3.92, 0.12
- V12 : balanced data set with slightly right skewed. Data range between -13, 15.08 and mean ~ 1.6 data concentrated between -0.4, 3.57



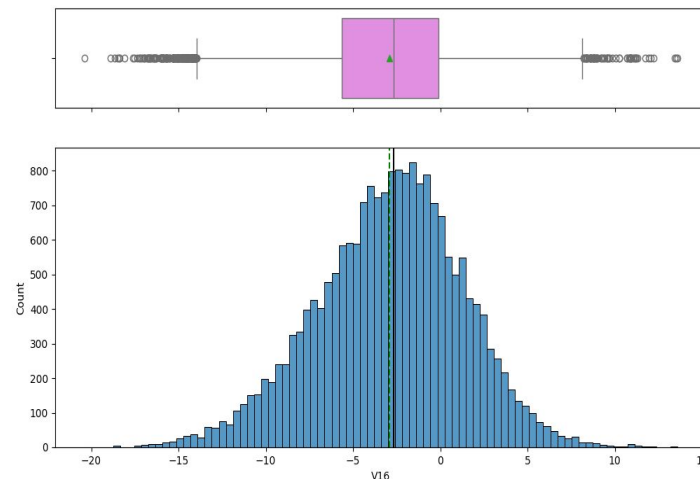
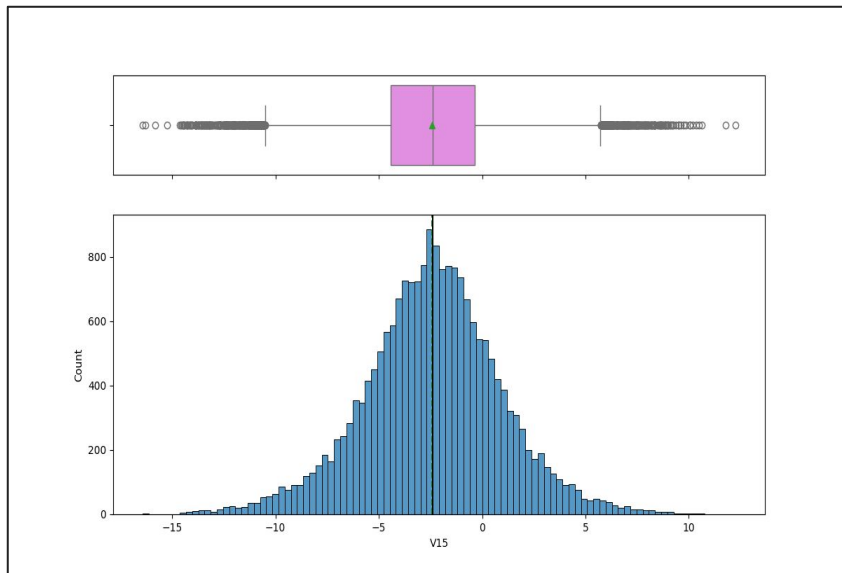
Data background check

- V13 : balanced data set is zero skewed and Data range between -13, 15.42 and mean ~ 1.58 data concentrated between -0.22, 3.46
- V14 : balanced data set is zero skewed and Data range between -8, 5.67 and mean ~ -0.95 data concentrated between -2.17, 0.27



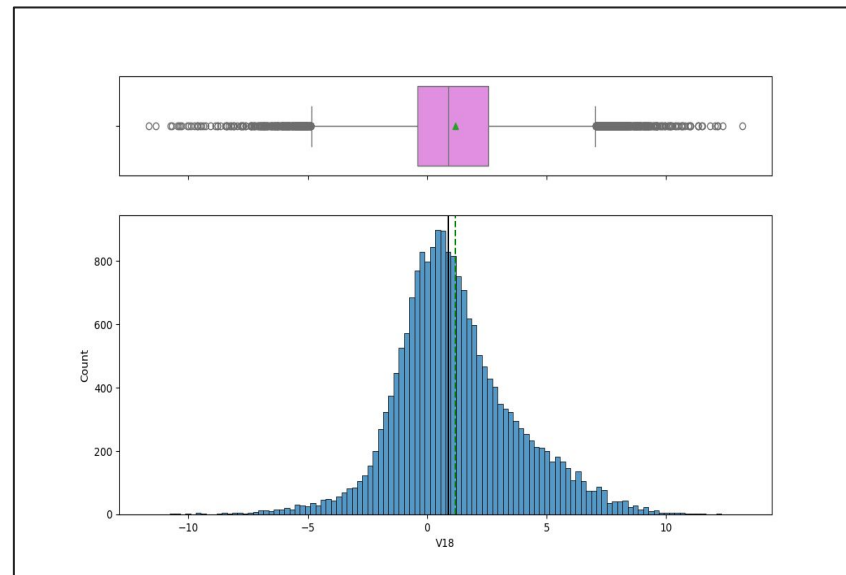
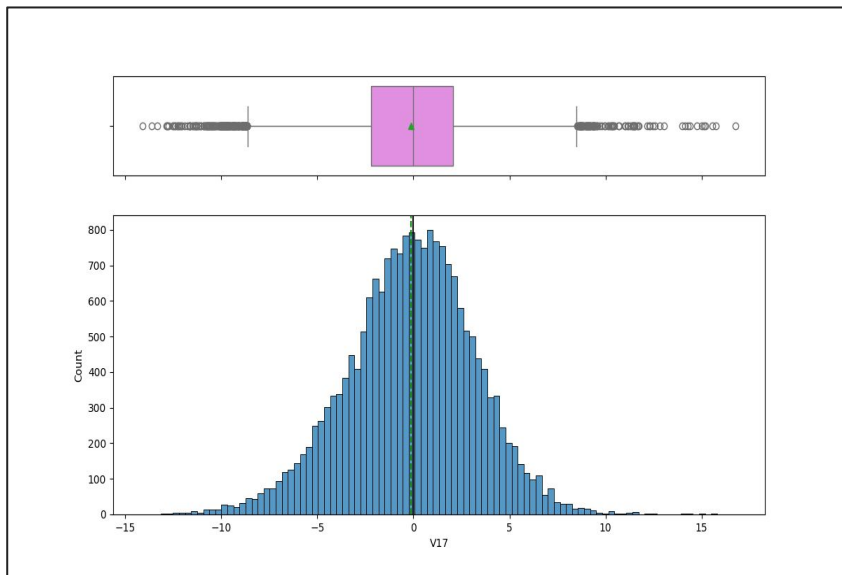
Data background check

- V15 : Balanced data set is zero and Data range between -16, 12.25 and mean ~ -2.41 data concentrated between -4.42, -0.36
- V16 : Data is slightly left skewed and Data range between -20, 13.58 and mean ~ -2.93 data concentrated between -5.63, -0.1



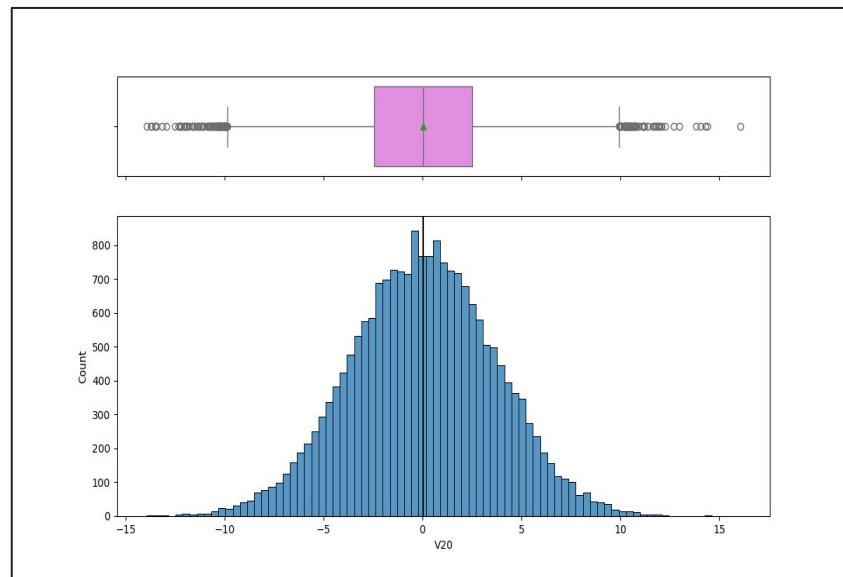
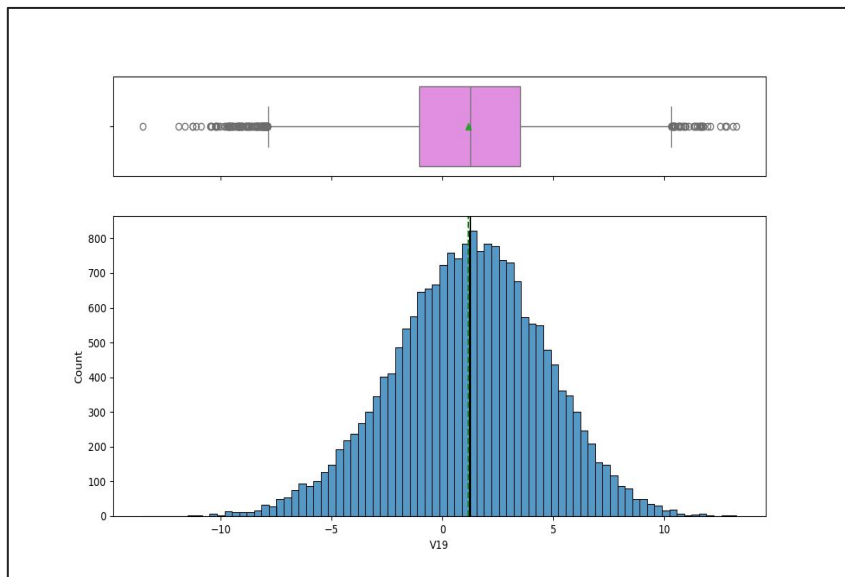
Data background check

- V17 : Data is left skewed and Data range between -14, 16.76 and mean ~ -0.13 data concentrated between -2.22, 2.07
- V18 : Data is right Skewed Data range between -12, 13.18 and mean ~ 1.19 data concentrated between -0.4, 2.57



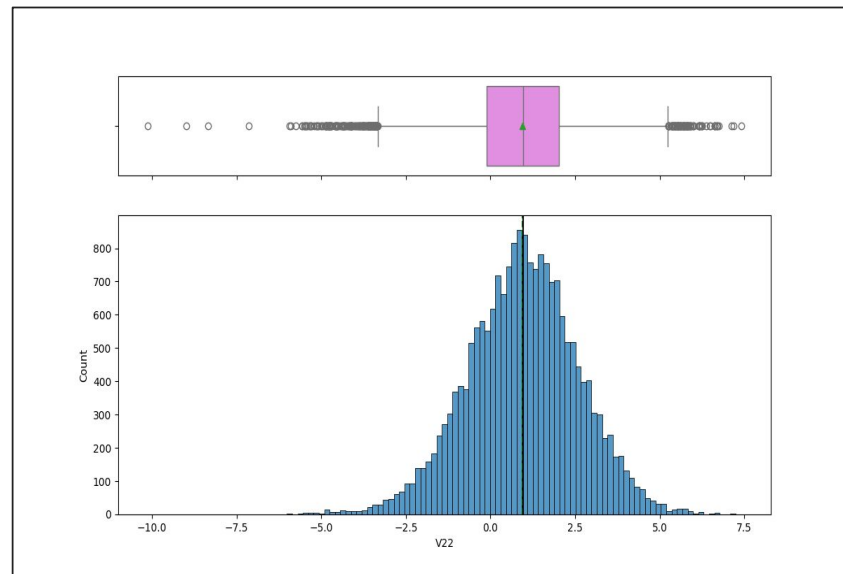
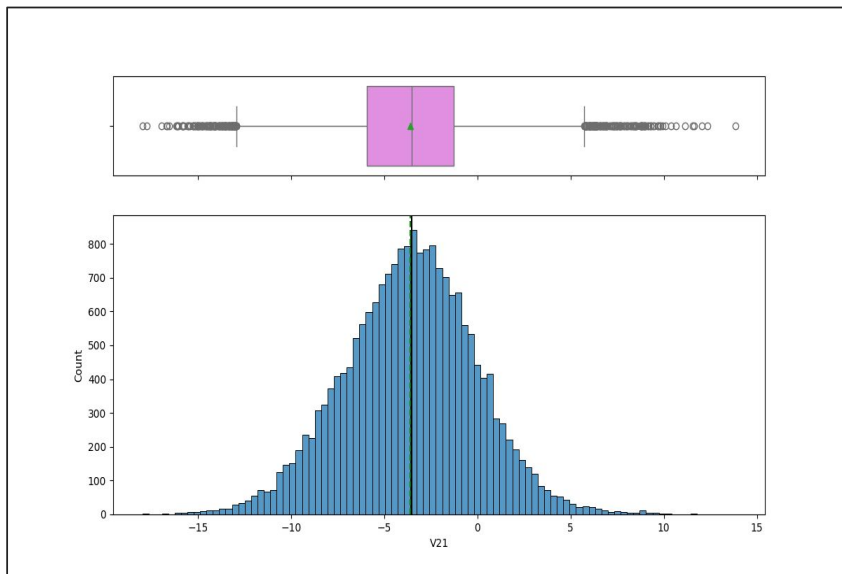
Data background check

- V19 : Data is slightly right skewed and Data range between -13, 13.24 and mean ~ 1.18 data concentrated between -1.05, 3.49
- V20 : Data is Left skewed Data range between -14, 16.05 and mean ~ 0.02 data concentrated between -2.43, 2.51



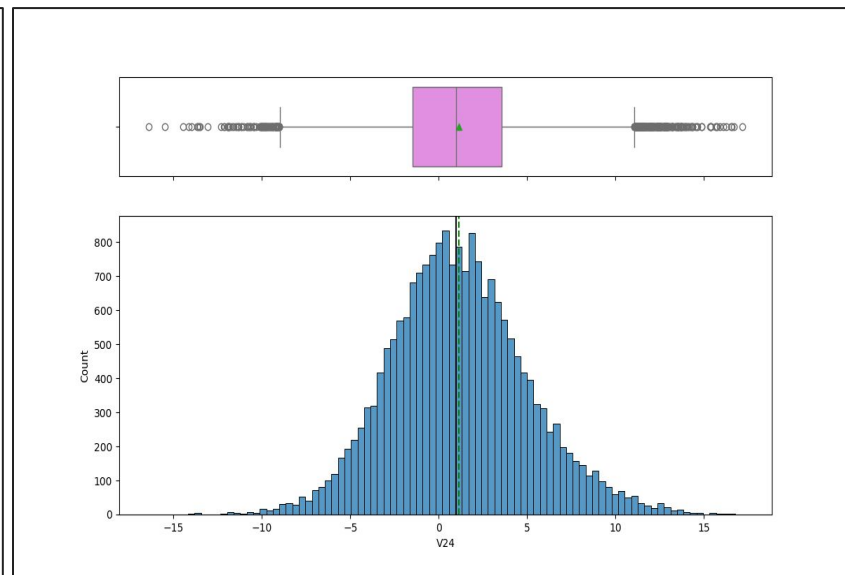
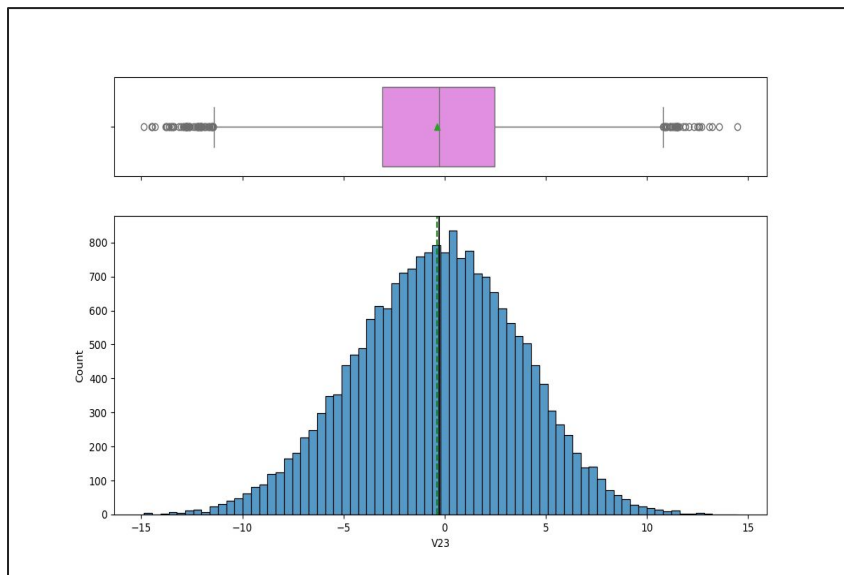
Data background check

- V21 : Data is balanced Data range between -18 and 13.84 and mean ~ -3.61 data concentrated between -5.93 and -1.27
- V22 : Data is slightly left skewed. Data range between -10 and 7.41 and mean ~ 0.95 data concentrated between -0.12 and 2.03



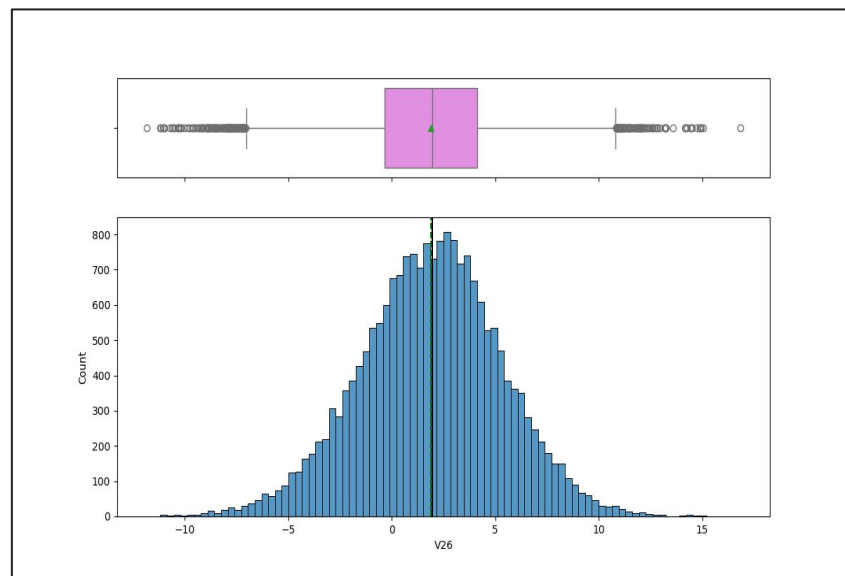
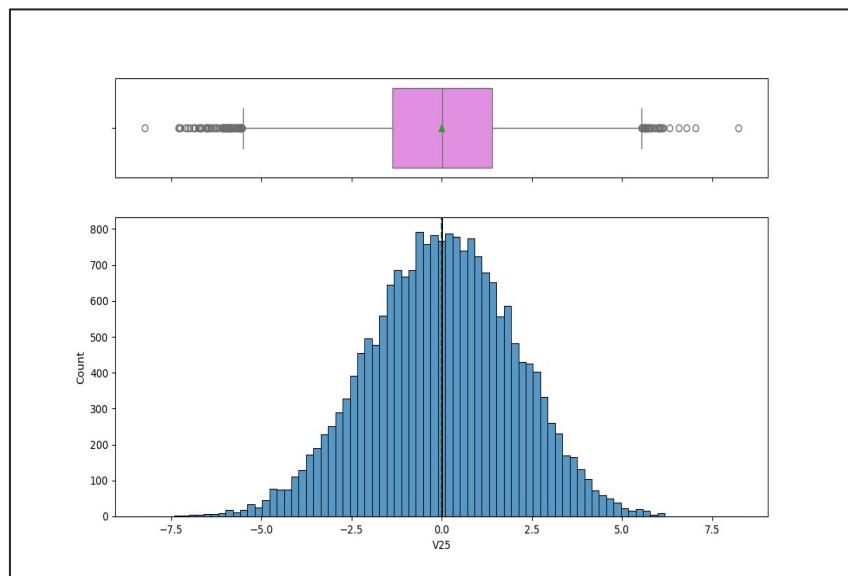
Data background check

- V23 : Data is slightly right skewed and Data range between -15, 14.46 and mean ~ -0.37 data concentrated between -3.1, 2.45
- V24 : Data is slightly left skewed and Data range between -16, 17.16 and mean ~ 1.13 data concentrated between -1.47, 3.55



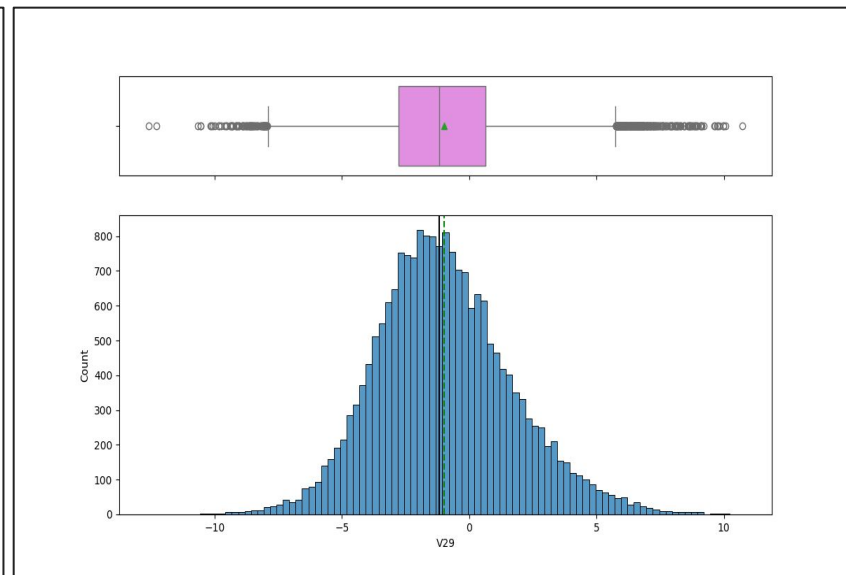
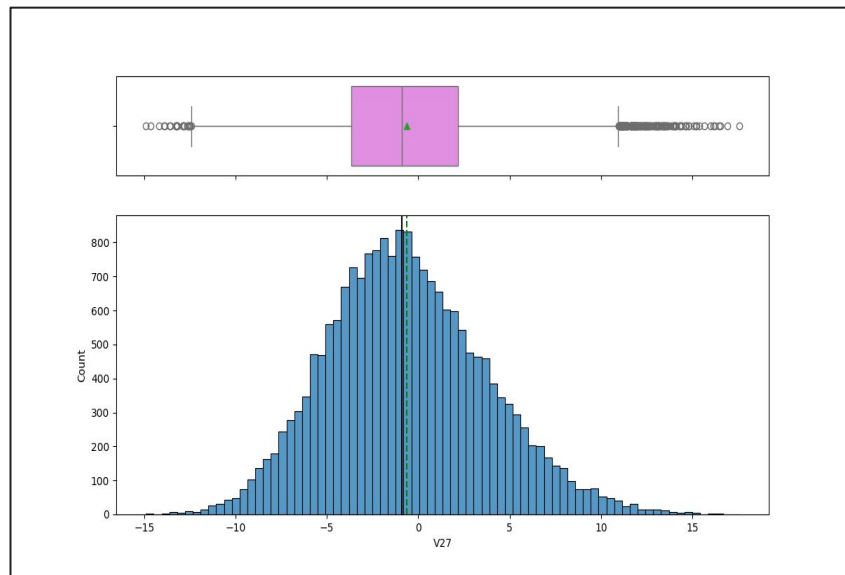
Data background check

- V25 : Data is balanced and 0 skewed. Data range between -8, 8.22 and mean ~0 data concentrated between -1.37, 1.4
- V26 : Data is Left skewed Data range between -12, 16.84 and mean ~1.87 data concentrated between -0.34, 4.13



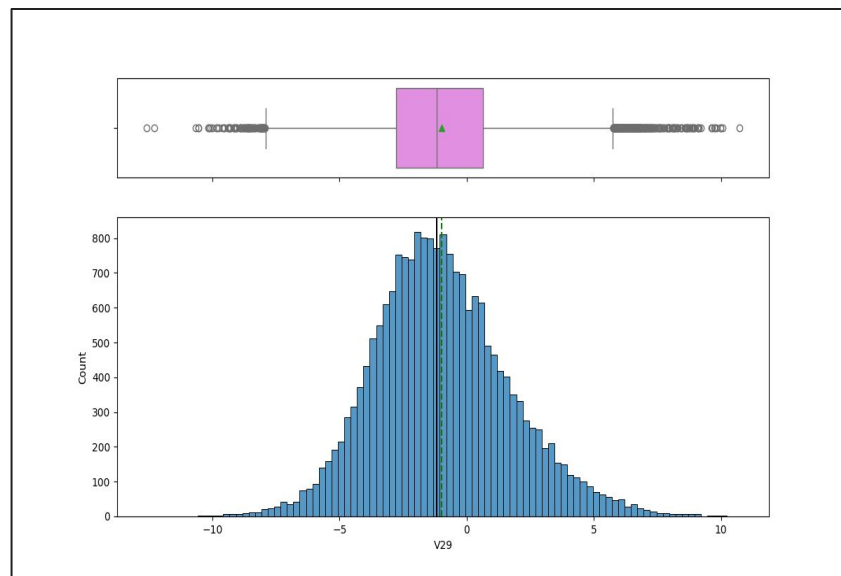
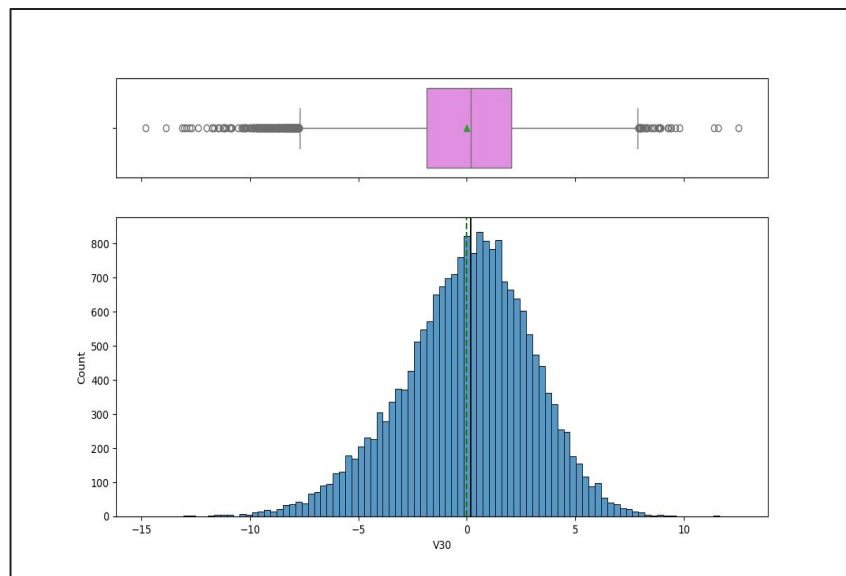
Data background check

- V27 : Data is right Skewed Data range between -15, 17.56 and mean ~ -0.61 data concentrated between -3.65, 2.19
- V28 : Data is right Skewed Data range between -9, 6.53 and mean ~ -0.88 data concentrated between -2.17, 0.38



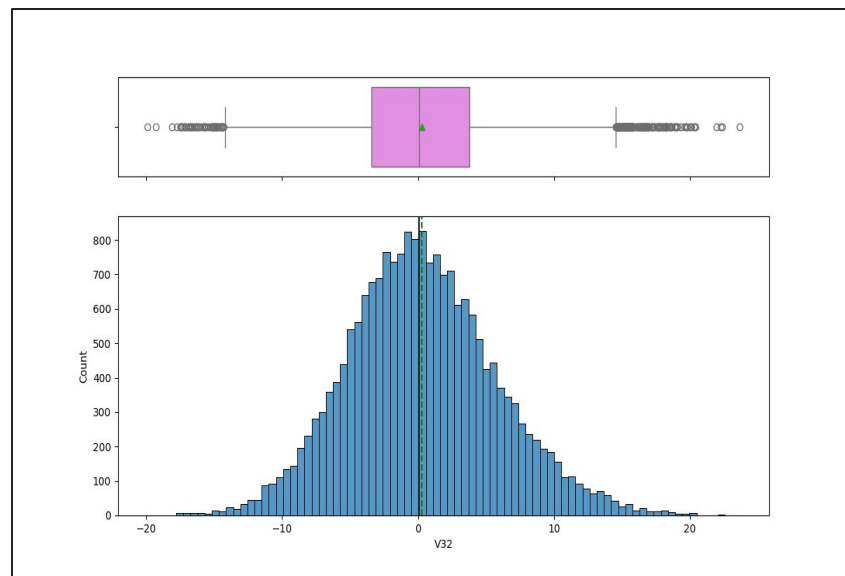
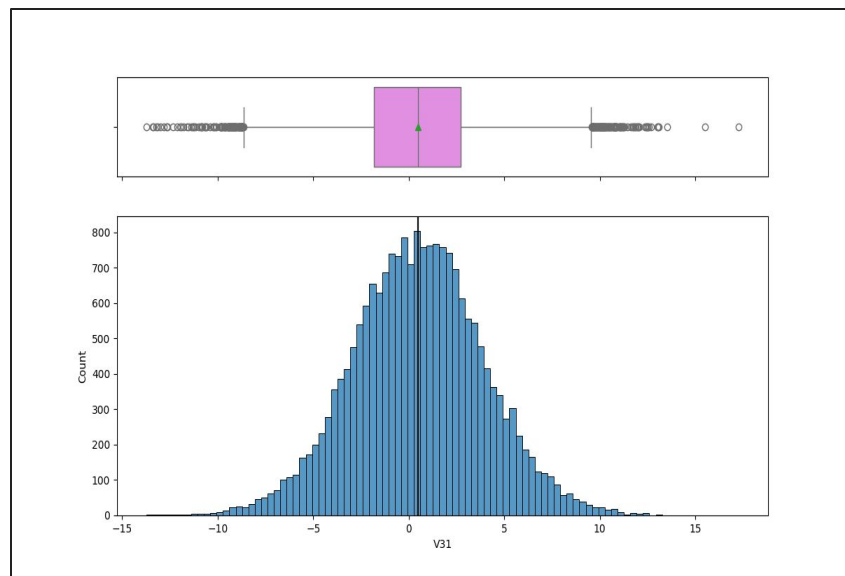
Data background check

- V29 : Data is right Skewed Data range between -13, 10.72 and mean ~ -0.99 data concentrated between -2.79, 0.63
- V30 : Data is Left skewed Data range between -15, 12.51 and mean ~ -0.02 data concentrated between -1.87, 2.04



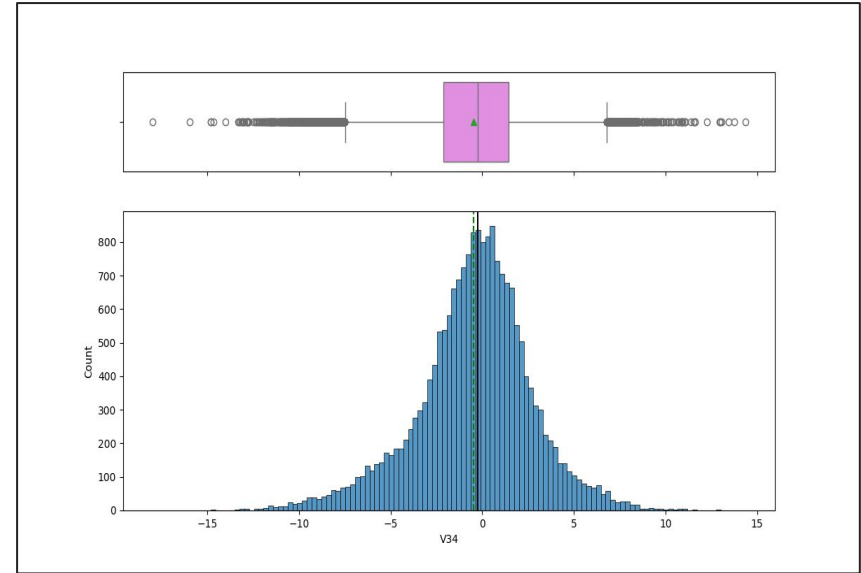
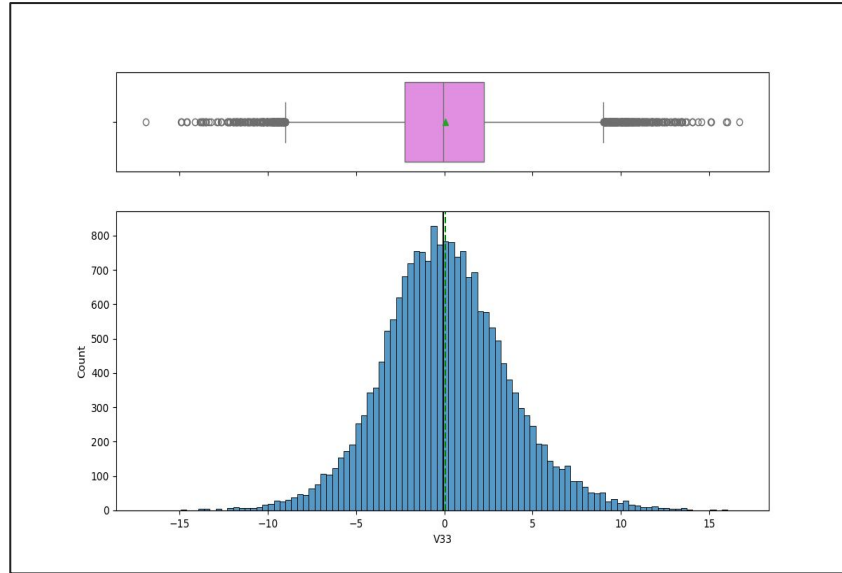
Data background check

- V31 : Data is Left skewed Data range between -14, 17.26 and mean ~ 0.49 data concentrated between -1.82, 2.73
- V32 : Data is right Skewed Data range between -20, 23.63 and mean ~ 0.3 data concentrated between -3.42, 3.76



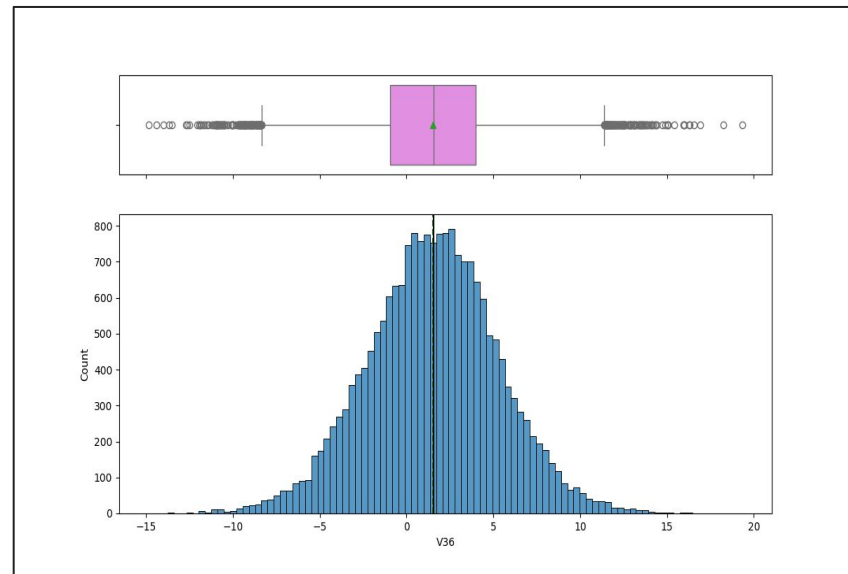
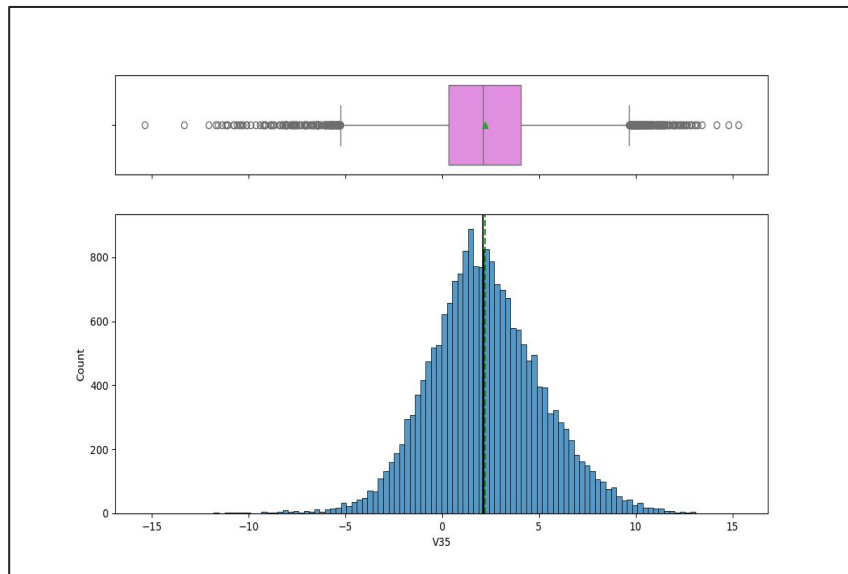
Data background check

- V33 : Data is balanced and 0 Skewed. Data range between -17, 16.69 and mean ~ 0.05 data concentrated between -2.24, 2.26
- V34 : Data is Left skewed. Data range between -18, 14.36 and mean ~ -0.46 data concentrated between -2.14, 1.44



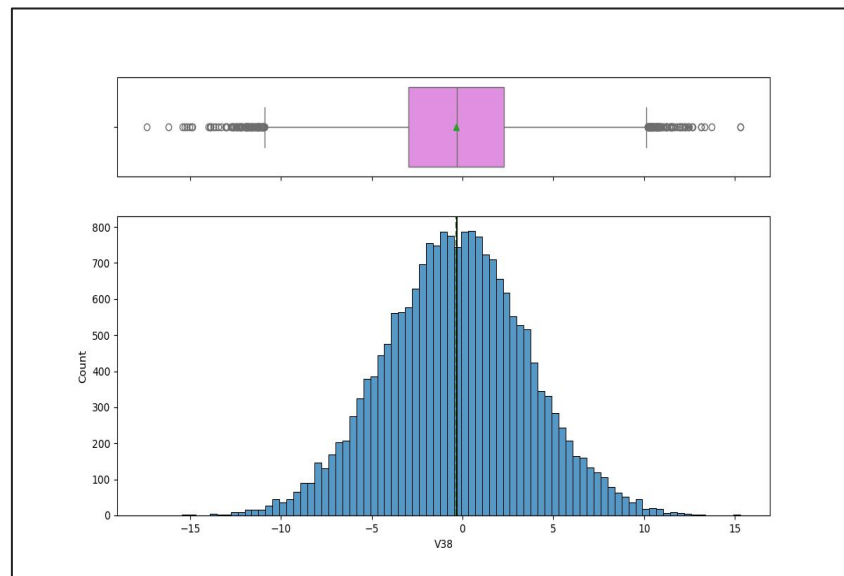
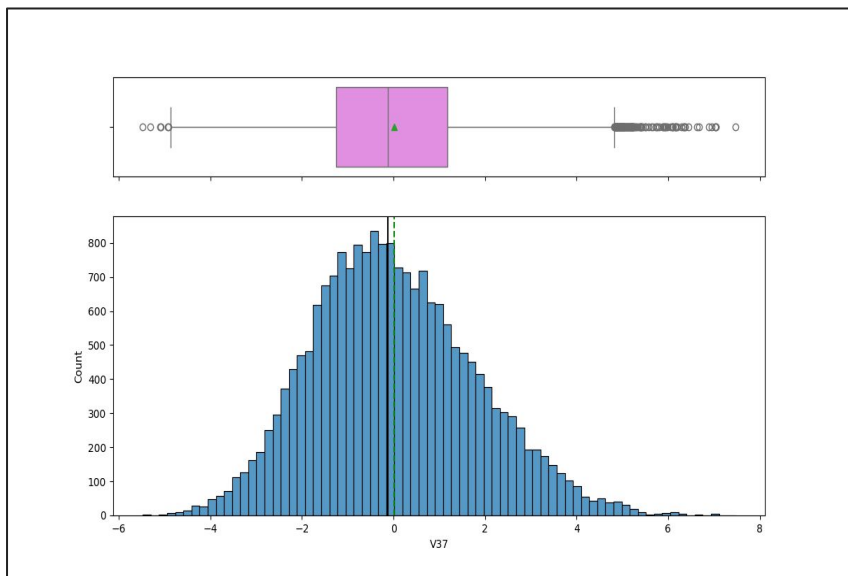
Data background check

- V35 : Data is right Skewed Data range between -15, 15.29 and mean ~2.23 data concentrated between 0.34, 4.06
- V36 : Data is Left skewed Data range between -15, 19.33 and mean ~1.51 data concentrated between -0.94, 3.98



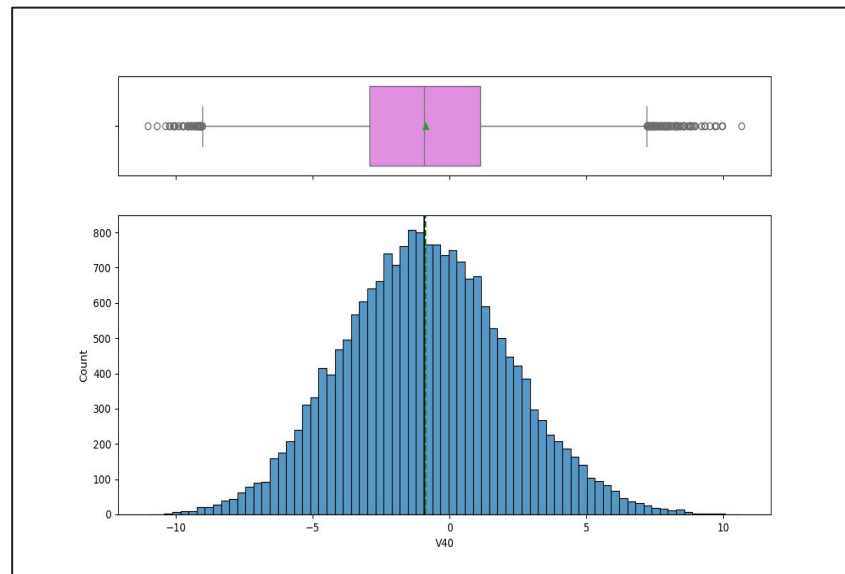
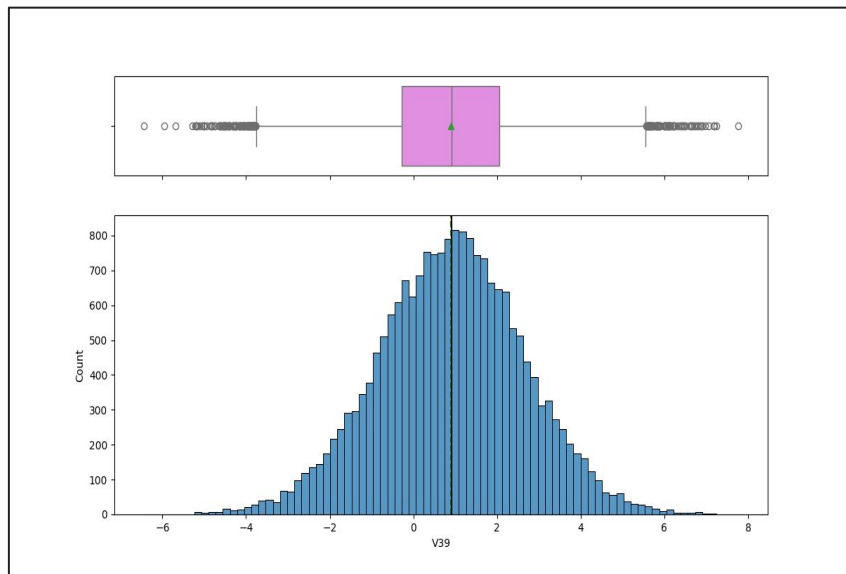
Data background check

- V37 : Data is right Skewed Data range between -5, 7.47 and mean ~ 0.01 data concentrated between -1.26, 1.18
- V38 : Data is Left skewed Data range between -17, 15.29 and mean ~ -0.34 data concentrated between -2.99, 2.28



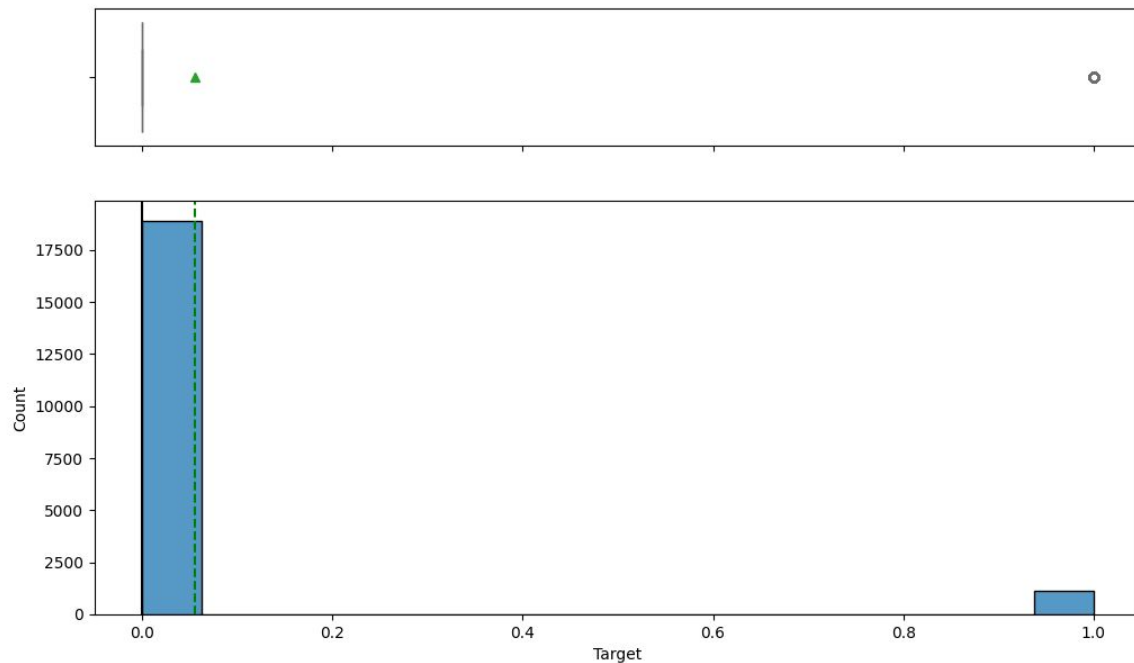
Data background check

- V39 : Data is Left skewed Data range between -6, 7.76 and mean ~ 0.89 data concentrated between -0.27, 2.06
- V40 : Data is right Skewed Data range between -11, 10.65 and mean ~ -0.88 data concentrated between -2.94, 1.12



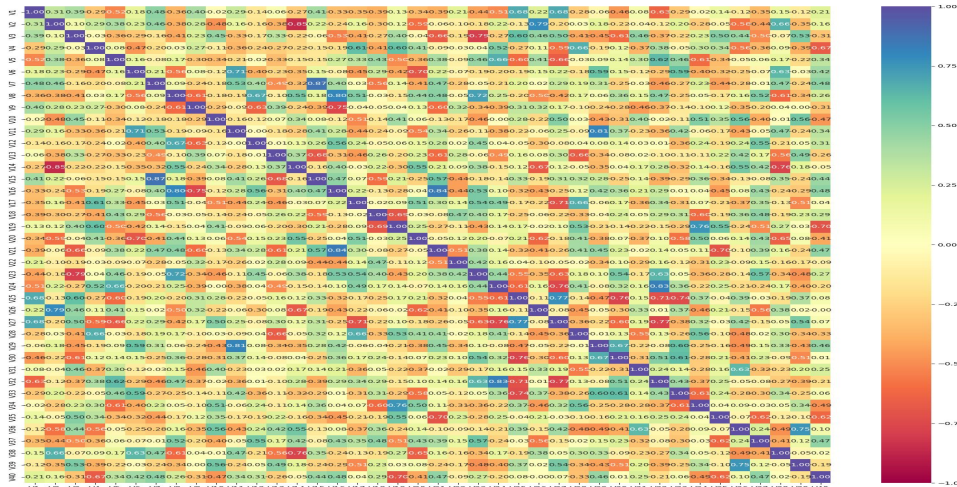
Data background check

Target : It is Binary classification feature and this value will be predicated using the ML Model Value. Data is very imbalance and value one occurs only 5.5% and rest of the data is 0



Data background check

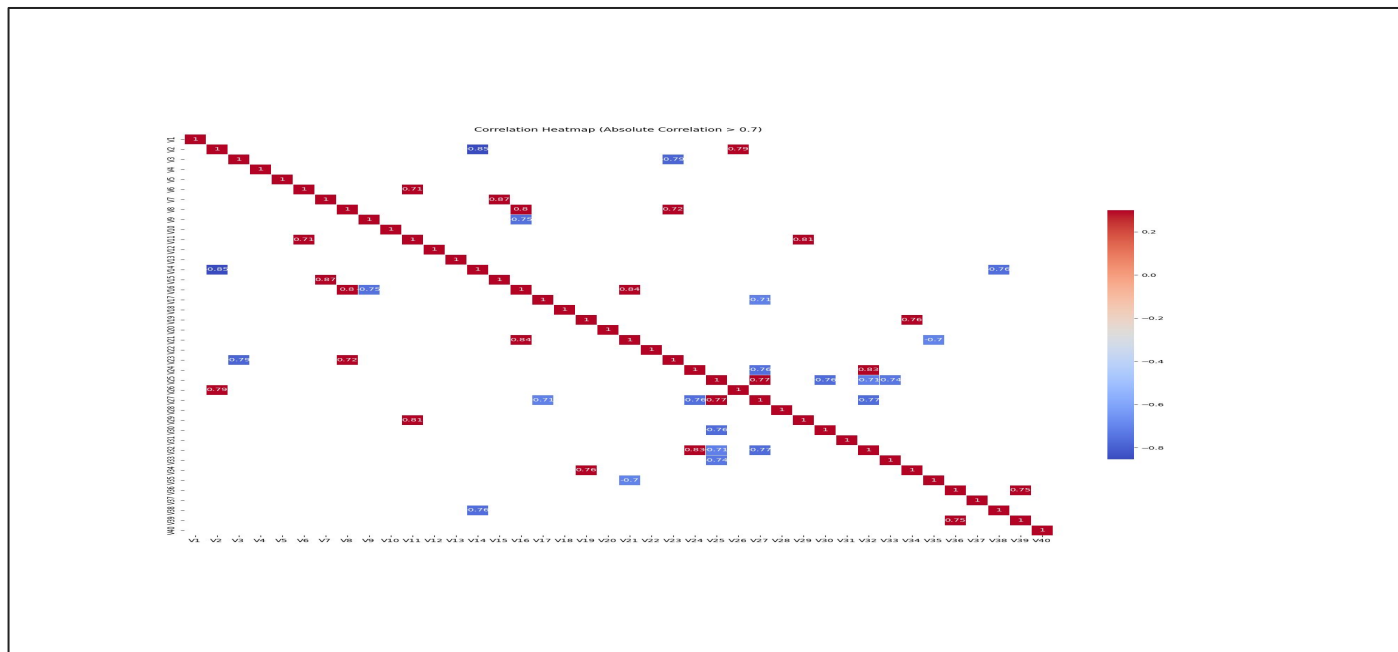
- Heat map



[Link to Appendix slide on data background check](#)

Data background check

- Heat map filtered for high correlation coefficient indicate some concentrated area between V24 to V32 and indicate related sensor area or signal



Model Trained and Tunned

- Model Architecture tried during training and tuning phase

Training Parameters	epoch = 150	batch = 500	loss='binary_crossentropy'	metrics = ['Recall']		
Model Name	Dense	Dropout	dense_1	dense_2	dense_3	Optimizer
Model 0	64. relu	-	1 sigmoid			SGD
Model 1	64. relu		32 relu	1 sigmoid		SGD
Model 2	64. relu	0.5	32 relu	16 relu	1 sigmoid	SGD
Model 3	64. relu	0.5	32 relu	16 relu	1 sigmoid	SGD + class balace (0:1.06, 1: 18.01)
Model 4	64 relu	-	32 relu	1 sigmoid		Adam
Model 5	64 relu	0.5	32 relu	16 relu	1 sigmoid	Adam
Model 6	64 relu	0.5	32 relu	16. relu	1 sigmoid	Adam + class_weight={0: 1.0, 1: 20.0}

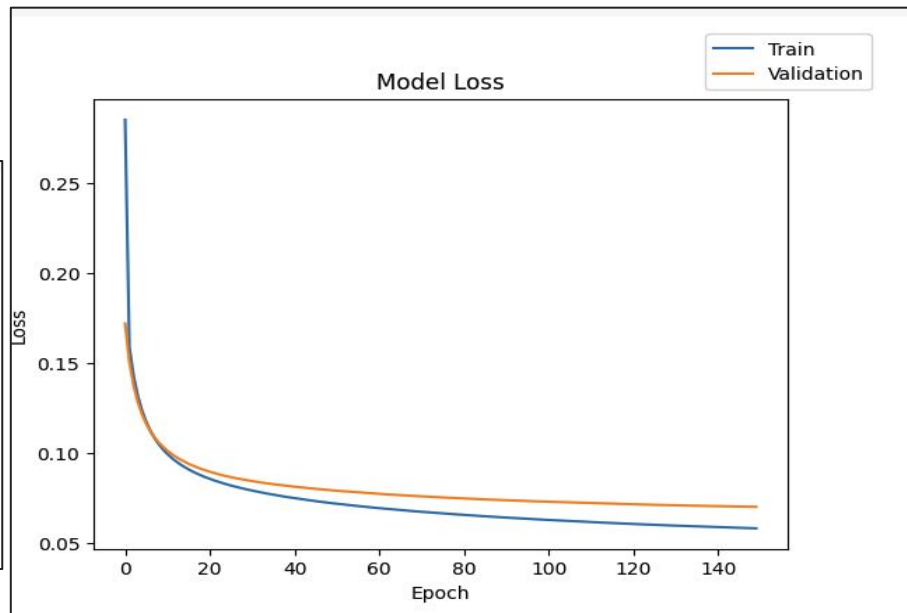
Model Trained and Tunned

- Model 0** base model started 40 input, single hidden layer of 64 neurons with Relu activation function 1 out put with sigmoid activation. SGD Optimizer is used. Both training and validation loss dropped significantly @ 15-17 epoch, The gap between training and validation loss is small, suggesting minimal overfitting. The validation curve flattens around epoch 100–120, which implies convergence.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dense_1 (Dense)	(None, 1)	65

Total params: 2,689 (10.50 KB)
 Trainable params: 2,689 (10.50 KB)
 Non-trainable params: 0 (0.00 B)



Model Trained and Tuned

- Model 0 well on class 0 with perfect recall score and near perfect f1 and precision score too, but it performed poorly on the class 1 and also quite drop in validation score from training indicate overfitting. It was a good starting point to iterate further

Classification Report - Train data Model_0				
	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	15112
1.0	0.97	0.82	0.89	888
accuracy			0.99	16000
macro avg	0.98	0.91	0.94	16000
weighted avg	0.99	0.99	0.99	16000

Classification Report - Validation data Model_0				
	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.95	0.79	0.86	222
accuracy			0.99	4000
macro avg	0.97	0.89	0.93	4000
weighted avg	0.99	0.99	0.99	4000

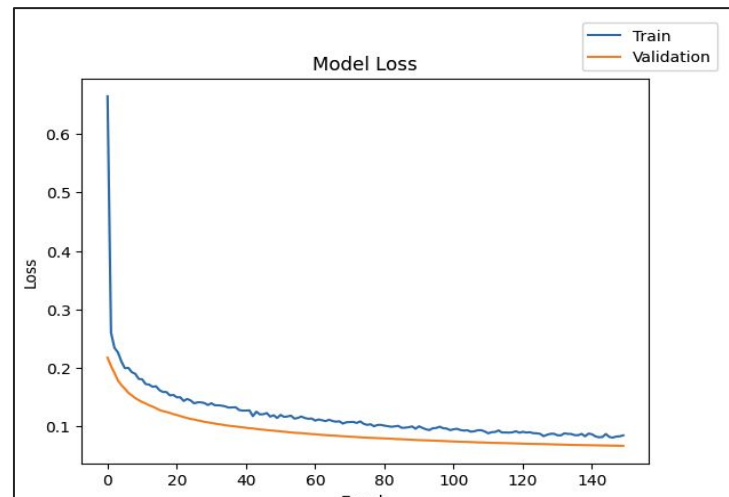
Model Trained and Tunned

- **Model 2** included an additional hidden layer. It used **40 input features**, followed by **three hidden layers** with **64, 32, and 16 neurons**, respectively, all using the **ReLU activation function**. It also used 50% dropout after the first hidden layer. The output layer consisted of **one neuron with a sigmoid activation**. The model was trained using the **SGD optimizer**.
- There's a visible loss drop till epoch 20, indicating the model learned important patterns early.
- After epoch ~40, both curves decrease gradually, showing continued but slower improvement.
- Around epoch 100–120, the validation loss flattens, indicating the model is converging
- The gap between training and validation loss is minimal and consistent, suggesting good generalization

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)
Trainable params: 5,249 (20.50 KB)
Non-trainable params: 0 (0.00 B)



Model Trained and Tuned

- Model 2: performance seems dropped considerably compare to earlier model on class 1 for recall and F1 score. Precision stayed well for both classes

Classification Report – Train data Model_2

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	15112
1.0	0.98	0.75	0.85	888
accuracy			0.99	16000
macro avg	0.98	0.87	0.92	16000
weighted avg	0.98	0.99	0.98	16000

Classification Report – Validation data Model_2

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.98	0.76	0.86	222
accuracy			0.99	4000
macro avg	0.98	0.88	0.92	4000
weighted avg	0.99	0.99	0.98	4000

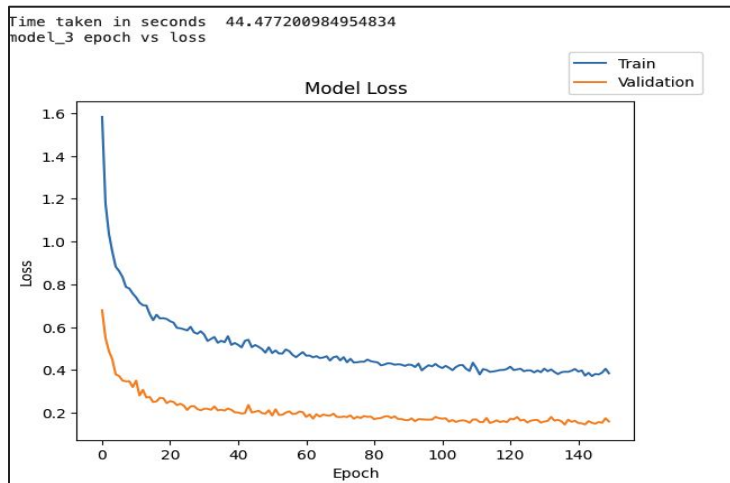
Model Trained and Tunned

- **Model 3** used same arch as model 2 but introduce class weight to give more weight to class zero in the ratio of (0:1, 0:18) It used **40 input features**, followed by **three hidden layers** with **64, 32, and 16 neurons**, respectively, all using the **ReLU activation function**. It also used 50% dropout after the first hidden layer. The output layer consisted of **one neuron with a sigmoid activation**. The model was trained using the **SGD optimizer**.
- There's a visible loss drop till epoch 20, indicating the model learned important patterns early.
- After epoch ~40, both curves decrease gradually, showing continued but slower improvement. But also observes some oscillation where losses are increased and then decreased.
- Model seems to be able to have better results on convergence on validation data set and generalized well

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)
Trainable params: 5,249 (20.50 KB)
Non-trainable params: 0 (0.00 B)



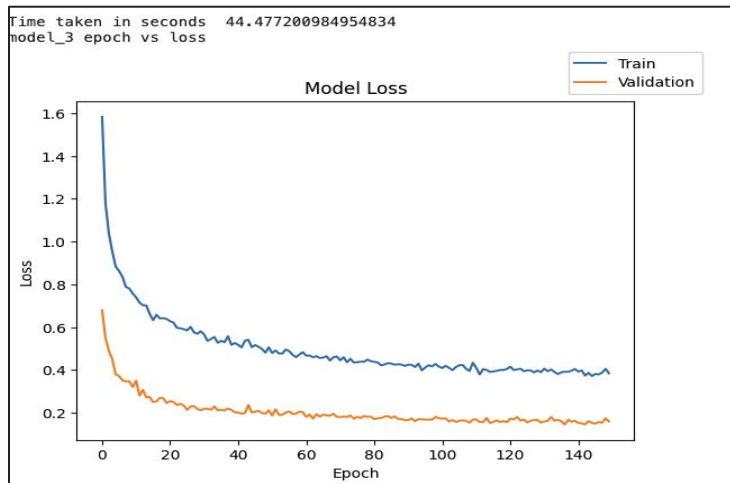
Model Trained and Tunned

- **Model 3** used same arch as model 2 but introduce class weight to give more weight to class zero in the ratio of (0:1, 0:18) It used **40 input features**, followed by **three hidden layers** with **64, 32, and 16 neurons**, respectively, all using the **ReLU activation function**. It also used 50% dropout after the first hidden layer. The output layer consisted of **one neuron with a sigmoid activation**. The model was trained using the **SGD optimizer**.
- There's a visible loss drop till epoch 20, indicating the model learned important patterns early.
- After epoch ~40, both curves decrease gradually, showing continued but slower improvement. But also observes some oscillation where losses are increased and then decreased.
- Model seems to be able to have better results on convergence on validation data set and generalized well

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)
Trainable params: 5,249 (20.50 KB)
Non-trainable params: 0 (0.00 B)



Model Trained and Tuned

- Model 3: Performed much better on class 0 compare to model 0, 1 and 2. On the recall (0.91) and F1 (0.90) score. It performed well and slight degradation on validation dataset but not significant one indicate good model and generalized well

Classification Report - Train data Model_3

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	15112
1.0	0.89	0.91	0.90	888
accuracy			0.99	16000
macro avg	0.94	0.95	0.95	16000
weighted avg	0.99	0.99	0.99	16000

Classification Report - Validation data Model_3

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	3778
1.0	0.90	0.89	0.89	222
accuracy			0.99	4000
macro avg	0.94	0.94	0.94	4000
weighted avg	0.99	0.99	0.99	4000

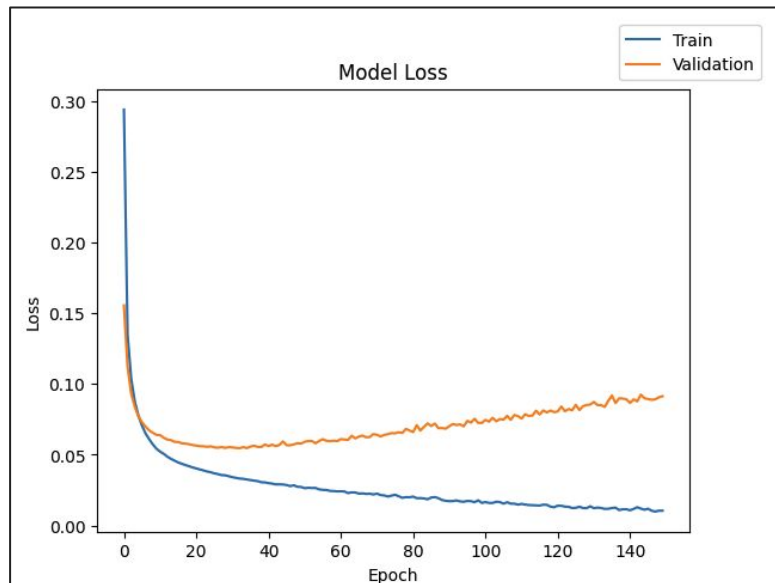
Model Trained and Tunned

- **Model 4** used **40 input features**, followed by **two hidden layers** with **64 and 32 neurons**, respectively, all using the **ReLU activation function**. The output layer consisted of **one neuron with a sigmoid activation**. The model was trained using the **Adam optimizer**.
- There's a visible loss drop till epoch 15, indicating the model learned important patterns early.
- Validation losses kept increasing after 15 epochs and seems model was not able to converge on validation set
- Training data set losses start converging steadily, indicate model overfit and learning noise along with the data

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 1)	33

Total params: 4,737 (18.50 KB)
Trainable params: 4,737 (18.50 KB)
Non-trainable params: 0 (0.00 B)



Model Trained and Tunned

- Model 4: performance seems dropped considerably compare Training set and also seems model is not a good fit Model got good recall score around .95 on train but Only 0.87 on the validation for class 0.

Classification Report – Train data Model_4

	precision	recall	f1-score	support
0.0	1.00	1.00	1.00	15112
1.0	1.00	0.95	0.98	888
accuracy			1.00	16000
macro avg	1.00	0.98	0.99	16000
weighted avg	1.00	1.00	1.00	16000

Classification Report – Validation data Model_4

	precision	recall	f1-score	support
0.0	0.99	1.00	0.99	3778
1.0	0.95	0.87	0.91	222
accuracy			0.99	4000
macro avg	0.97	0.93	0.95	4000
weighted avg	0.99	0.99	0.99	4000

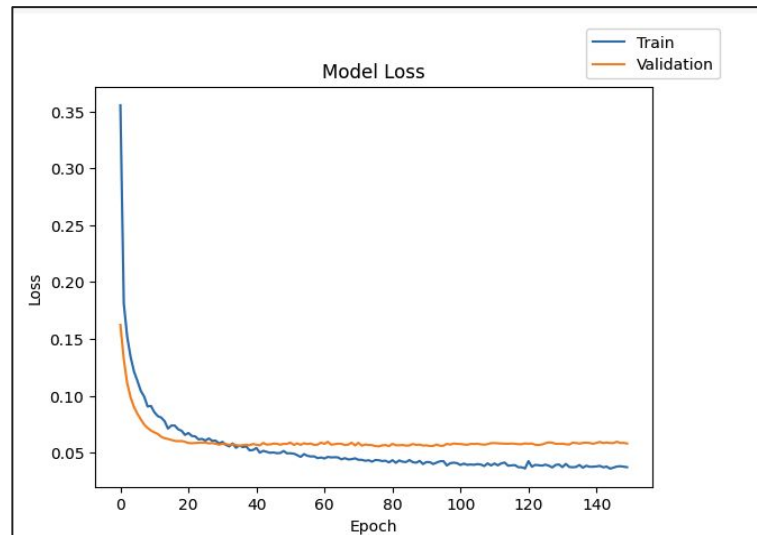
Model Trained and Tunned

- **Model 5** used a similar arch as model 2, **40 input features**, followed by **three hidden layers** with **64, 32, and 16 neurons**, respectively, all using the **ReLU activation function**. It also used 50% dropout after the first hidden layer. The output layer consisted of **one neuron with a sigmoid activation**. The model was trained using the **Adam optimizer**.
- There's a visible loss drop till epoch 15, indicating the model learned important patterns early.
- After epoch ~30, Validation losses didn't converge as fast as Training data set to indicate overfitting but better than Model 4

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)
Trainable params: 5,249 (20.50 KB)
Non-trainable params: 0 (0.00 B)



Model Trained and Tuned

- Model 5: performed well but lower than model 3 recall score for class one 1 is around 0.87 which is similar to model 2 but
- Model got better f1 score than all other models

Classification Report - Train data Model_5

	precision	recall	f1-score	support
0.0	0.99	1.00	1.00	15112
1.0	0.99	0.90	0.94	888
accuracy			0.99	16000
macro avg	0.99	0.95	0.97	16000
weighted avg	0.99	0.99	0.99	16000

Classification Report - Validation data Model_5

	precision	recall	f1-score	support
0.0	0.99	1.00	1.00	3778
1.0	0.98	0.87	0.92	222
accuracy			0.99	4000
macro avg	0.99	0.93	0.96	4000
weighted avg	0.99	0.99	0.99	4000

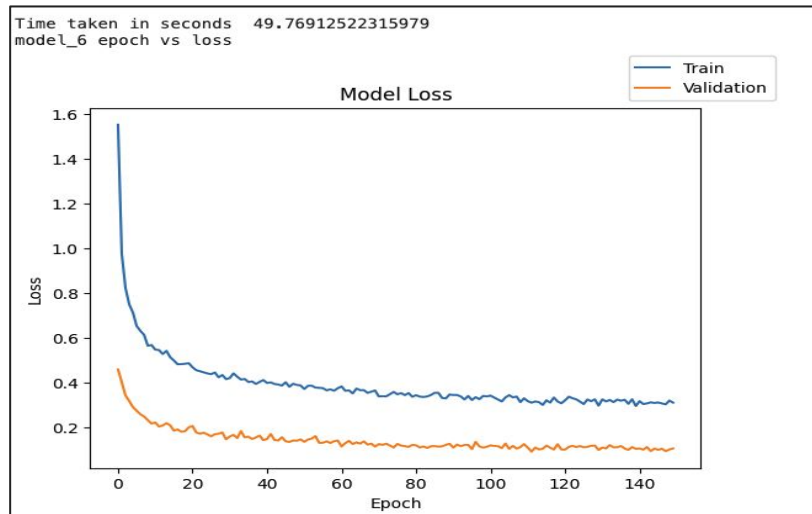
Model Trained and Tunned

- **Model 6** used similar arch as model 5 with class weight balance of (0:1, 1:20) **40 input features**, followed by **three hidden layers** with **64, 32, and 16 neurons**, respectively, all using the **ReLU activation function**. It also used 50% dropout after the first hidden layer. The output layer consisted of **one neuron with a sigmoid activation**. The model was trained using the **Adam optimizer**.
- There's a visible loss drop till epoch 15, indicating the model learned important patterns early.
- Model slowly start converging and become almost flat after 100 with some minor oscillation in loss values. models generalized well

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	2,624
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 5,249 (20.50 KB)
Trainable params: 5,249 (20.50 KB)
Non-trainable params: 0 (0.00 B)



Model Trained and Tuned

- Model 6: performed well, with quite good recall (0.90) on class 1. Minimal drop in validation score indicates model fit well It got generalized well.

Classification Report - Train data Model_6

	precision	recall	f1-score	support
0.0	1.00	0.99	0.99	15112
1.0	0.82	0.92	0.87	888
accuracy			0.98	16000
macro avg	0.91	0.95	0.93	16000
weighted avg	0.99	0.98	0.98	16000

Classification Report - Validation data Model_6

	precision	recall	f1-score	support
0.0	0.99	0.99	0.99	3778
1.0	0.82	0.90	0.85	222
accuracy			0.98	4000
macro avg	0.90	0.94	0.92	4000
weighted avg	0.98	0.98	0.98	4000

Model Assumptions

1. Input Feature Quality

- Assumes the **40 input features** are meaningful and contribute to the target prediction.
- Features are expected to be **normalized or scaled**, especially when using Adam (although it's more tolerant than SGD).
- Assumes **16,000 rows** are enough to train a neural network with **3 hidden layers** without overfitting.
- Overfitting risk is mitigated with **50% dropout** and **class weighting**.

Independence of Observations

- Like most ML models, assumes **each training sample is independent** of the others. This is critical for the loss function (e.g., binary cross-entropy) and stochastic gradient descent to function properly.

No Strong Multicollinearity

- While NNs can *technically* learn through multicollinearity, redundant features may still **confuse training**, slow convergence, or make interpretation difficult.
accept the black-box nature and rely on model performance, not interpretability



Happy Learning !

